

Tab 8

Hero Section Variations:

Hero Section Variations:

Here are all the image prompts we iterated through in this session, compiled in one place:

HRO-01 — Valley with Solar Panels + Sunlight

json

```
{
  "prompt_id": "HRO-01",
  "use_case": "Hero section background – Renewable Energy brand (Hulax-style landing page)",
  "subject": {
    "description": "A sweeping aerial drone shot over a lush, emerald-green valley at soft golden hour, with large-scale solar panel arrays deployed symmetrically across the valley floor.",
    "focal_elements": {
      "primary": "Rows of deep blue photovoltaic solar panels arranged in wide grid formations across the mid-ground, catching warm directional sunlight with a subtle blue-silver sheen",
      "secondary": "Rich emerald-green grass and rolling terrain surrounding the solar arrays",
      "tertiary": "Distant mountain ridgeline fading into atmospheric haze, with two or three wind turbines barely visible on the horizon"
    }
  },
  "scene": {
    "environment": "Outdoor – vast open green valley, aerial perspective at roughly 35–45 degree drone angle",
    "elements": "Gentle rolling hills frame the solar farm. A narrow winding dirt road cuts through the lower frame. Wispy clouds partially diffuse sunlight creating soft god rays. No human figures, no urban elements."
  },
  "lighting": {
    "setup": "Soft golden hour sunlight from upper-right at a low angle, casting long gentle shadows from the solar panels across the grass.",
  }
}
```

```
    "details": "Warm amber-gold highlights on solar panel frames. Cool blue tone on panel faces reflecting the sky. Green meadow glows with warm, saturated vitality.",
  },
  "camera": {
    "technical": "Drone aerial, wide-angle 16–24mm equivalent, f/5.6, golden hour, photorealistic commercial photography.",
    "constraints": [
      "16:9 landscape aspect ratio",
      "Solar panels clearly identifiable and prominent in mid-ground",
      "Sunlight soft and natural – NOT harsh midday",
      "Bottom-left area uncluttered for headline text overlay",
      "No people, no buildings, no urban infrastructure"
    ]
  },
  "mood": "Hopeful, expansive, clean energy at scale – harmony of human innovation and the natural world"
}
```

HRO-02 — Wind Turbines Only, No Solar Panels

json

```
{
  "prompt_id": "HRO-02",
  "use_case": "Hero section background – Renewable Energy brand, no solar panels",
  "subject": {
    "description": "A breathtaking aerial drone shot over a vast, lush green valley with wind turbines standing tall across the rolling hillsides, soft golden sunlight filtering through the atmosphere.",
    "focal_elements": {
      "primary": "Five to seven large white wind turbines distributed naturally across mid-ground and background hills, blades mid-rotation catching warm sunlight",
      "secondary": "Richly saturated emerald-green rolling hills with natural terrain variation – elevated ridgelines, low gentle meadows creating depth",
    }
  }
}
```

```
    "tertiary": "Distant mountain range softening into pale
blue-green atmospheric haze, bright diffused sky above"
  },
  "scene": {
    "environment": "Outdoor – wide open green highland valley, aerial
drone at approximately 40-degree downward angle, golden hour",
    "elements": "Narrow winding road visible diagonally in lower-mid
frame. No buildings, no people. Occasional patches of low morning mist
in the lower valley. Light wispy clouds letting golden rays through."
  },
  "lighting": {
    "setup": "Soft golden hour sunlight from upper-right, diffused
through thin cloud cover. Long soft shadows trail behind wind turbines
across the hillside.",
    "details": "Warm golden highlights on turbine blades and hilltops.
Cool shadow pockets in lower valley folds. Scene feels luminous –
sunlight breaking through, not blazing."
  },
  "camera": {
    "technical": "Drone aerial, 16–24mm equivalent, f/5.6, golden
hour, photorealistic high-resolution commercial style.",
    "constraints": [
      "16:9 landscape aspect ratio",
      "Wind turbines clearly visible and monumental in scale",
      "Soft warm golden hour light – NOT harsh midday",
      "Bottom-left quadrant clean and low-contrast for text overlay",
      "No solar panels, no buildings, no people, no vehicles"
    ]
  },
  "mood": "Vast, hopeful, and quietly powerful – the unstoppable
momentum of clean energy in harmony with the natural world"
}
```

HRO-03 — Three Turbines + Sunlight Falling on Ground

json

```
{
  "prompt_id": "HRO-03",
  "use_case": "Hero section background – Renewable Energy brand,
updated sunlight and turbine count",
  "subject": {
    "description": "Aerial drone shot over a vast lush emerald-green
valley with exactly three large wind turbines in the foreground and
mid-ground only. Warm golden sunlight visibly falls and pools on the
ground, concentrated toward the bottom-left of the frame.",
    "focal_elements": {
      "primary": "Exactly three large white wind turbines – one in
near foreground, two in mid-ground. No turbines on distant ridgelines
or background mountains.",
      "secondary": "Visible sunlight falling on the ground – warm
golden pools of light illuminating the green valley floor, especially
sweeping across the bottom-left area",
      "tertiary": "Rich textured emerald-green rolling hills with
ridgelines catching full sunlight, lower valley folds in softer
shadow"
    }
  },
  "scene": {
    "environment": "Outdoor – wide open green highland valley, aerial
drone at approximately 40-degree downward angle, golden hour",
    "elements": "Narrow winding road in the lower-mid frame. No
buildings, no people. Soft morning mist in lower valley. Sun
positioned upper-right sending dramatic light shafts downward toward
the bottom-left, physically landing and spreading across the ground.",
    "lighting_on_ground": "CRITICAL – sunlight must be visibly cast
onto the ground surface, not just glowing in the sky. Bright warm
golden patches of light fall across the green valley floor, most
intensely in the bottom-left quadrant."
  },
  "lighting": {
    "setup": "Golden hour sun positioned upper-right, partially behind
thin clouds, sending two to three broad god-ray light shafts
diagonally downward from upper-right to lower-left, physically
illuminating the ground.",
```

```
    "details": "Strong specular warm highlight along top edges of turbine blades. Valley floor in bottom-left shows brightest ground-level illumination – warm amber-gold light pooling on the grass. Shadow pockets in valley folds for tonal depth.",
  },
  "camera": {
    "technical": "Drone aerial, 16-24mm equivalent, f/5.6, golden hour, photorealistic high-resolution commercial photography. High dynamic range.",
    "constraints": [
      "16:9 landscape aspect ratio",
      "EXACTLY three wind turbines – foreground and mid-ground only, NONE in far background",
      "Sunlight MUST visibly fall and pool on the ground – especially bottom-left",
      "God rays visible traveling upper-right to lower-left",
      "No solar panels, no buildings, no people, no vehicles",
      "Three turbines maximum – none behind the mid-ground"
    ]
  },
  "mood": "Radiant, expansive, and quietly powerful – golden sunlight blessing the land, clean wind energy standing tall"
}
```

HRO-04 — Three Turbines + Vibrant Greens + Orange Sunlight Glow (Latest)

json

```
{
  "prompt_id": "HRO-04",
  "use_case": "Hero section background – Renewable Energy brand, vibrant greens with warm orange sunlight glow",
  "subject": {
    "description": "Aerial drone shot over a vast, intensely vibrant and fresh emerald-green valley with exactly three large wind turbines in the foreground and mid-ground only. Greens remain deeply saturated
```

and lush. A warm orangish-golden sunlight glow is layered over the landscape, sitting on top of the green without washing it out.",

```
"focal_elements": {  
  "primary": "Exactly three large white wind turbines – one in  
near foreground, two in mid-ground. No turbines in far background.",  
  "secondary": "Fresh, deeply saturated emerald and lime-green  
rolling hills – green MUST remain vivid and full of life, not yellowed  
or desaturated by the sunlight",  
  "tertiary": "Warm orangish-golden sunlight glow washing softly  
across terrain – most intense toward the bottom-left where light  
visibly pools on the ground surface"  
},  
},  
"scene": {  
  "environment": "Outdoor – wide open green highland valley, aerial  
drone at approximately 40-degree downward angle",  
  "elements": "Narrow winding road in lower-mid frame. No buildings,  
no people. Soft morning mist lightly in lower valley folds. Sun  
positioned upper-right, sending warm orangish light shafts diagonally  
toward the bottom-left – light physically landing and spreading across  
the green ground.",  
  "lighting_on_ground": "CRITICAL – warm orange-gold sunlight must  
be visibly cast and pooling on the ground surface in the bottom-left  
quadrant. Light patches glow with warm amber-orange tone against vivid  
green grass."  
},  
"lighting": {  
  "setup": "Late golden hour sun upper-right, diffused through thin  
clouds, casting two to three broad warm orange-tinted god rays  
diagonally from upper-right down toward lower-left ground. Orange  
color temperature – warmer than standard golden hour, closer to  
late-day amber-orange.",  
  "details": "KEY RULE: GREENS STAY GREEN. Orange light sits on top  
of green terrain as complementary color contrast. Turbine blades catch  
warm orange rim highlights. Valley floor bottom-left shows strongest  
warm orange ground pooling. Shadow areas in valley folds remain cool  
deep green."  
},  
}
```

```
"camera": {
  "technical": "Drone aerial, 16-24mm equivalent, f/5.6, late golden
hour, photorealistic high-resolution commercial photography. Vibrant
color grading – high green saturation retained throughout.",
  "constraints": [
    "16:9 landscape aspect ratio",
    "EXACTLY three wind turbines – foreground and mid-ground only,
NONE in far background",
    "Green saturation MUST remain high and vibrant – fresh electric
emerald and lime greens throughout",
    "Sunlight color temperature must be warm orange-amber – NOT cool
white or pale yellow",
    "Sunlight MUST visibly fall and pool on the ground –
concentrated toward bottom-left",
    "God rays traveling diagonally upper-right to lower-left must be
visible",
    "Orange glow and green terrain must coexist – neither overpowers
the other",
    "No solar panels, no buildings, no people, no vehicles",
    "Three turbines maximum, all in near-to-mid distance"
  ]
},
"color_interaction": {
  "rule": "Orange sunlight on vivid green terrain – complementary
color contrast. Warm orange glow should ENHANCE the perceived
vividness of the greens, not replace them. Greens pushed to +20
saturation, light source dialed to warm 3200K orange temperature.",
  "reference_feel": "Similar to aerial footage of Irish or New
Zealand highlands shot at magic hour – intensely green land glowing
under a warm amber sky"
},
"mood": "Radiant vitality – unstoppable energy of nature meeting
clean technology, warm sunlight blessing a deeply green and living
landscape",
"brand_alignment": {
  "color_palette": [
    "#1B6B18 (deep shadow valley green)",
    "#4CAF1A (mid-tone vibrant hill green)",
```

```
    "#A8E832 (bright lit lime-green hilltop)",
    "#FF8C00 (warm orange sunlight glow)",
    "#FFB347 (amber-orange ground light pool)",
    "#FFFFFF (turbine white with warm rim)"
  ],
  "avoid": "Desaturated greens, olive or yellow-green tones, harsh
white midday light, cool-toned sunlight, muted cinematic grades, more
than three turbines, turbines in far background"
}
```

Quick evolution summary:

ID	Key Change
HRO-01	Solar panels + golden hour, no turbine count limit
HRO-02	Removed solar panels, added 5–7 turbines, clean valley
HRO-03	Capped to 3 turbines, added god rays + sunlight on ground
HRO-04	Kept HRO-03 base, added orange sunlight glow, enforced vibrant greens stay green

Image Prompts:

Here are all **5 structured JSON image prompts** for every image generated in this conversation:



HERO IMAGES

Prompt 01 — Hero V1: Golden Hour Solar Farm

json

```
{
  "prompt_id": "PVG-HERO-01",
  "use_case": "Hero section background – animate as slow time-lapse
with moving clouds",

  "subject": {
    "description": "A breathtaking wide-angle cinematic aerial
photograph of a large-scale solar panel farm at golden hour, captured
from a low camera angle close to the ground looking across hundreds of
rows of monocrystalline solar panels stretching into the horizon.",
    "focal_elements": {
      "primary": "Hundreds of rows of dark navy monocrystalline solar
panels receding into the distance in perfect geometric perspective
lines, glass faces glowing with reflected amber-gold light",
      "secondary": "The setting sun sitting just above the horizon
line, partially obscured by dramatic layered cumulus clouds, casting
long golden light across the panel surfaces",
```

```
    "tertiary": "Long dark shadows between each panel row creating a strong rhythmic geometric pattern across the ground"
  },
},
```

```
"scene": {
  "environment": "Open agricultural flatland converted into a utility-scale solar farm, with low wild grass and wildflowers along the perimeter fencing",
  "sky": "Highly dramatic – deep cerulean blue at the zenith blending through violet to fiery amber-gold at the horizon. Scattered cumulus clouds with dark purple-grey bases and brilliantly lit magenta-orange tops. Volumetric crepuscular light rays cutting through gaps in clouds near the horizon.",
  "elements": "Long grass at the foreground edge, subtle bird silhouettes mid-sky, rows of panels with metallic sheen catching the last warm light of the day",
  "animation_potential": "Clouds are soft-edged and naturally shaped – ideal for slow rightward drift via Ken Burns or parallax scroll animation"
},
```

```
"lighting": {
  "setup": "Hard directional golden hour sunlight from low on the horizon at approximately 10 degrees above ground – creating maximum specular highlights on panel glass and maximum shadow contrast between rows",
  "details": "Warm 2700K amber color temperature throughout the lower scene. Panels glow with reflected amber light. Sky retains cool 6500K blues in upper portion creating a dramatic warm-cool contrast split."
},
```

```
"camera": {
  "technical": "Shot on Phase One IQ4 medium format camera, 32mm wide lens, f/11, ISO 64. Ultra-high-resolution cinematic color grading. Photorealistic.",
}
```

```
    "angle": "Ground-level perspective approximately 1 meter above
ground, looking across panel rows toward the horizon",
    "constraints": [
        "Aspect ratio 16:9 landscape – ultra-wide cinematic",
        "Ground-level perspective looking across panel rows",
        "Clouds MUST be visible and dramatic – key for animation",
        "No people or text in frame",
        "Maximum cinematic depth with leading lines from panel rows
converging at horizon"
    ]
},

    "mood": "Epic, inspiring, warm, clean energy optimism – premium
magazine cover quality",
    "negative_prompt": "Avoid: flat sky, overcast dull lighting, people,
text, vehicles, artificial objects, HDR over-processing, painterly
style"
}
```

Prompt 02 — Hero V2: Revised (3 Turbines + God Rays on Ground)

```
json
{
    "prompt_id": "PVG-HERO-02-REVISED",
    "use_case": "Hero section background – animate as slow time-lapse
with drifting clouds and sweeping light beams",

    "subject": {
        "description": "A breathtaking wide-angle cinematic landscape of a
solar panel farm spread across lush rolling green hills at golden
hour. Exactly three large modern white wind turbines are placed within
the scene – no more, no less. No wind turbines in the distant
background. The scene is dominated by golden diagonal sunlight rays
visibly streaming from the upper-left and landing on the ground.",
        "focal_elements": {
```

```
    "primary": "Rows of dark navy-blue monocrystalline solar panels arranged in neat parallel lines across the rolling green hillside, glass faces catching warm amber-gold light from the upper-left",
    "secondary": "Exactly three tall modern white wind turbines – one large and prominent in the center-left foreground, one slightly smaller center-right mid-ground, one partially cropped at the far right edge. NO turbines anywhere in the background or horizon.",
    "tertiary": "Dramatic volumetric crepuscular light rays (god rays) streaming diagonally from the upper-left corner of the frame at 40-45 degrees, visibly landing as warm golden light pools on the green grass ground between panel rows and on the panel surfaces themselves"
  },
  "lighting": {
    "setup": "The sun is positioned completely OFF-FRAME to the upper-left – never visible within the image. This off-frame sun position creates the diagonal god rays slicing from upper-left to lower-right across the entire scene.",
    "ground_light": "CRITICAL – sunlight beams visibly LAND on and illuminate the ground. Warm brilliant gold patches of light fall across: (1) green grass strips between solar panel rows, (2) solar panel glass surfaces, (3) hillside slopes in mid-ground. These lit patches contrast strongly against cooler blue-green shadowed areas.",
    "shadow_contrast": "Strong shadow-to-light contrast. Areas between beams are cooler blue-green tone. Areas where beams land glow in warm amber-gold. Dappled light effect across the ground is the signature visual.",
    "color_temperature": "Warm 2700K golden amber in all lit areas. Cool 6500K blue-green in all shadowed areas. High contrast between two zones is essential."
  },
  "scene": {
    "environment": "Rolling lush green hills similar to New Zealand, Irish, or German countryside. Hills undulate gently giving panel rows a slightly curved organic layout. Tall wild grass and low shrubs visible between panel rows.",
```

```
"sky": "Dramatic and active – large billowing cumulus clouds with deep purple-grey flat bases and brilliantly gold-lit tops and edges. Clouds are soft-edged, naturally shaped, positioned unevenly for animatability.",
  "wind_turbine_placement": {
    "turbine_1": "Largest – center-left foreground, rising prominently above solar panel rows. Blades in slight motion blur from rotation.",
    "turbine_2": "Medium – center-right mid-ground, slightly behind turbine 1. Same motion blur on blades.",
    "turbine_3": "Smallest apparent size – far right of frame, base partially behind a hill or panel row. Blades visible against sky.",
    "rule": "STRICTLY NO OTHER TURBINES anywhere in the image – not in background, not on horizon, not blurred in distance. Only these three."
  },
  "animation_potential": "Clouds drift slowly rightward. God rays sweep or pulse in intensity. Grass subtly sways. Panel reflections shift as light angle changes in time-lapse sequence."
},

"camera": {
  "technical": "Shot on Phase One IQ4 150MP with Schneider-Kreuznach 32mm LS lens, f/11, ISO 64. Tripod mounted. Photorealistic cinematic quality.",
  "angle": "Ground-level, camera approximately 1 meter above ground, looking slightly upward at 5-10 degrees to emphasize turbine height against sky",
  "constraints": [
    "Aspect ratio STRICTLY 16:9 landscape",
    "EXACTLY THREE wind turbines – count must be verifiable",
    "NO wind turbines in background or distant horizon",
    "God rays MUST originate from upper-left and visibly land on ground",
    "Sunlight patches on GROUND between panel rows are MANDATORY",
    "Sun must be OFF-FRAME upper-left – never visible in image",
    "Lush green rolling hills – not flat terrain",
    "No people, no text, photorealistic only"
```

```
    ],
  },
  "mood": "Epic, cinematic, warm, powerful, hopeful – premium clean energy landscape that feels alive and dynamic",
  "negative_prompt": "Avoid: more than 3 wind turbines, flat terrain, overcast dull sky, desert landscape, oversaturated HDR look, painterly style, sun visible in frame, god rays that do not reach ground, uniform flat lighting, any people or vehicles"
}
```

BENTO BOX IMAGES

Prompt 03 — Bento: Hybrid Solar Inverter Product Shot

```
json
{
  "prompt_id": "PVG-BENTO-01",
  "use_case": "Bento grid card – Inverters and Optimizers product category",

  "subject": {
    "description": "A sleek modern hybrid solar inverter and charge controller unit mounted on a clean white-painted concrete wall inside a bright minimal utility room. Photographed perfectly straight-on as a centered product hero shot.",
    "focal_elements": {
      "primary": "Inverter unit in matte black and brushed anodized aluminum finish – rectangular form factor approximately 50cm wide x 40cm tall, centered on the wall",
      "secondary": "Small high-contrast LCD display on the front face showing live output wattage – 4,200W – with a simple energy flow diagram and status readout",
      "tertiary": "Row of green LED status indicators along the right edge of the unit. Color-coded DC input cables – red positive, black
```

```
negative – entering from below through a clean cable management
channel"
    },
    "scene": {
        "environment": "Clean minimal utility room – smooth white concrete
wall, light oak flooring visible at bottom, one recessed ceiling LED
above casting even light. Absolutely no clutter.",
        "elements": "Small cable conduit at the bottom of the unit. A
product specification label on the lower-right corner of the unit. A
single small green LED glowing on the unit face.",
        "animation_potential": "LCD screen number can be animated counting
up in real time. LED indicators can pulse slowly on loop."
    },
    "lighting": {
        "setup": "Perfectly even soft box studio lighting from both left
and right sides simultaneously – mimicking professional tech product
photography",
        "details": "Crisp white highlights on brushed aluminum edges. Very
subtle warm fill from below to show underside cable detail. Clean drop
shadow behind unit against wall surface."
    },
    "camera": {
        "technical": "Shot on Sony A7R V with 85mm f/2.8 lens. Perfectly
flat front-on angle. ISO 100. Commercial product photography quality.
Photorealistic.",
        "constraints": [
            "Square 1:1 aspect ratio",
            "Perfectly centered symmetrical composition",
            "White background wall MUST be clean and bright",
            "LCD display MUST show readable wattage numbers",
            "No text overlays – product hardware only",
            "No people, no props other than the inverter and wall"
        ]
    },
}
```

```
    "mood": "Clean tech, premium hardware, trusted German engineering, catalog quality",
    "negative_prompt": "Avoid: cluttered background, multiple products, dark moody lighting, any people, overly dramatic shadows, wide-angle lens distortion"
}
```

Prompt 04 — Bento: Monocrystalline Solar Cell Macro Close-Up

json

```
{
  "prompt_id": "PVG-BENTO-02",
  "use_case": "Bento grid card – Our Technology section or product detail visual",

  "subject": {
    "description": "An extreme close-up macro photograph of a monocrystalline solar cell surface shot at a low 15-degree grazing angle across the cell face, revealing the stunning precision engineering of the silver busbars, finger grid lines, and the iridescent anti-reflective silicon coating.",
    "focal_elements": {
      "primary": "Grid of ultra-thin silver metallic finger lines running in perfectly parallel rows across the deep navy-black silicon surface – filling the entire frame",
      "secondary": "Three wider horizontal busbars crossing the finger lines perpendicularly with a subtle bright metallic sheen catching the directional studio light",
      "tertiary": "An iridescent anti-reflective coating creating a stunning rainbow sheen of deep blue, violet, and teal across the silicon cell face – more vivid in the foreground, softly blurred in the background"
    }
  },
}
```

```
"scene": {
  "environment": "No background visible – the solar cell surface
fills the entire frame. Shallow depth of field with the foreground
cells razor sharp and background cells softly dissolving into bokeh.",
  "elements": "Tiny dewdrops or condensation pearls at the very top
edge of the frame, each one refracting a miniature rainbow of
prismatic light. The silicon surface texture is fully revealed by the
raking light.",
  "animation_potential": "Works as a slow parallax zoom-in or
pull-back for scroll animation. The bokeh background creates natural
depth layering."
},

"lighting": {
  "setup": "Single hard directional raking studio light from 20
degrees off the panel surface – grazing the cell face to reveal
maximum surface texture and make the silver grid lines pop
brilliantly",
  "details": "Cool specular highlights on silver finger lines.
Iridescent teal-blue-violet sheen across the silicon. Soft fill light
from the opposite side retaining shadow detail without crushing
blacks."
},

"camera": {
  "technical": "Canon MP-E 65mm macro lens on Canon R5, f/5.6, 1:1
magnification, multiple focus-stacked exposures combined. ISO 100.
Commercial macro product photography quality.",
  "constraints": [
    "Square 1:1 aspect ratio",
    "Entire frame filled with cell surface – no white background, no
borders",
    "Silver grid lines MUST be razor sharp in center frame",
    "Iridescent blue-violet-teal sheen MUST be visible across
silicon surface",
    "Dewdrops at top edge are optional but strongly preferred",
    "Low grazing angle across the surface – not straight down"
  ]
}
```

```
    },  
  
    "mood": "Technical precision, premium engineering, scientific  
beauty, microscopic wonder",  
    "negative_prompt": "Avoid: flat overhead lighting, white background,  
no texture visible, dark flat silicon with no sheen, soft focus  
throughout"  
}
```

Prompt 05 — Bento: Complete Solar System Components Flat Lay

```
json  
{  
  "prompt_id": "PVG-BENTO-03",  
  "use_case": "Bento grid card – Shop overview, product catalog, or  
homepage featured products section",  
  
  "subject": {  
    "description": "A premium top-down flat-lay product photograph of  
all major solar panel system components neatly arranged on a clean  
bright white marble surface. Each component is spaced evenly and  
clearly identifiable as a distinct product.",  
    "focal_elements": {  
      "primary": "One full monocrystalline solar panel placed  
diagonally at the top-left – dark navy cell matrix, silver aluminum  
frame, clearly identifiable as the centerpiece of the arrangement",  
      "secondary": "A compact hybrid inverter unit in brushed aluminum  
at top-right. A neat coiled bundle of red and black DC solar cables  
with MC4 connectors at center-left. A set of silver anodized aluminum  
roof mounting rails and clamps at center.",  
      "tertiary": "A small lithium iron phosphate battery module with  
blue casing at bottom-right. A compact solar charge controller with  
LED indicators at bottom-left. Each item has clear breathing room  
around it – no overlapping."  
    }  
  },  
}
```

```
"scene": {
  "environment": "Perfectly clean bright white marble surface with
very subtle natural veining. Shot from directly overhead at a perfect
90-degree bird's-eye top-down angle. No background distractions.",
  "elements": "Each component casts a very soft subtle shadow
directly beneath it from the perfectly even studio overhead lighting.
Components are arranged with deliberate graphic balance – heaviest
items at corners, cables in the center as visual connectors.",
  "animation_potential": "Components can be animated to slide into
frame from different directions and snap into final position – ideal
for a product showcase animation sequence"
},

"lighting": {
  "setup": "Perfectly even large-area diffused studio lighting from
directly above – zero harsh shadows, zero hot spots. Every component
is equally lit with clean soft light.",
  "details": "Each component shows its true material quality:
aluminum shows brushed grain, solar panel glass shows slight
translucent depth, cables show rubber texture, battery casing shows
matte plastic texture."
},

"camera": {
  "technical": "Shot on Phase One 80mm lens directly overhead at 90
degrees – perfectly flat bird's-eye view. f/11, ISO 50. Commercial
product flat-lay photography quality. Ultra-sharp throughout entire
frame.",
  "constraints": [
    "Square 1:1 OR landscape 4:3 aspect ratio",
    "Perfectly straight overhead top-down angle – no perspective
distortion",
    "Pure white marble background – bright and clean",
    "ALL components must be fully visible and not overlapping",
    "Every component must be in sharp focus throughout the entire
frame",
    "No people, no hands, no text labels in the image"
```

```

    ],
    "mood": "Premium product unboxing, Apple-style clean composition, complete system overview, aspirational purchase",
    "negative_prompt": "Avoid: dark background, overlapping products, any single product cropped out of frame, depth of field blur on any component, hands or people, props or decorative items unrelated to solar systems"
}

```

All 5 Prompts — Master Reference Table

Prompt ID	Image	Section Use	Ratio	Key Visual
PVG-HERO-01	Hero V1	Hero background	16:9	Golden hour solar farm, dramatic sky
PVG-HERO-02-REVISED	Hero V2	Hero background	16:9	3 turbines, god rays landing on ground
PVG-BENTO-01	Inverter	Product bento card	1:1	Inverter on white wall, LCD display
PVG-BENTO-02	Solar macro	Tech bento card	1:1	Iridescent silicon cell, silver grid lines
PVG-BENTO-03	Components flatlay	Shop bento card	1:1	All 5 components on white marble

All prompts are ready to paste directly into **Midjourney**, **Flux**, **Adobe Firefly**, **Stable Diffusion**, **DALL·E 3**, or **Google Imagen** for high-resolution production renders. Want me to add **negative prompts** or **style reference weights** for any specific tool?

Review



The Definitive One-Shot Master Prompt: Photovoltaic Goldstein Website

Author's Note: This is an incredibly massive, multi-page strict integration prompt designed for an elite Large Language Model. It contains the exact HTML structure, strict CSS variables, Liquid Glass component logic, and the GSAP/Canvas JavaScript required to recreate the high-end Photovoltaic Goldstein e-commerce website on the first try from zero-to-one. Do not invent classes. Do not use React or Tailwind. Execute exactly as directed.

1. Project Overview & Architectural Directives

Objective: Write the continuous raw code (

index.html,

styles.css,

main.js) for a premium German solar technology storefront. **Aesthetic Baseline:** "Precision Engineering" meets "Apple-level minimal luxury." You will heavily rely on "Liquid Glass" (blur/backdrop-filters over dark themes), massive bold serif typography for hero sections, and smooth 30fps butter-like animations.

Strict Technical Constraints:

1. **HTML Structure:** Semantic HTML5 only. Avoid excessive nesting.
2. **CSS:** Pure vanilla CSS. Use modern CSS Variables heavily. Absolutely NO Tailwind CSS or utility-class frameworks.
3. **JavaScript:** Pure Vanilla JS. Avoid jQuery.
4. **Animation Libraries (via CDN):**
 - GSAP Core (`gsap.min.js`)
 - GSAP ScrollTrigger (`ScrollTrigger.min.js`)
 - GSAP Draggable (`Draggable.min.js`)
 - GSAP Inertia (`InertiaPlugin.min.js`)
5. **E-Commerce Engine:** Shopify Storefront Web Components (CDN). No liquid templates.
6. **Icons:** Lucide Icons (`unpkg.com/lucide@latest`).
7. **Fonts:** `<link>` loaded via Google Fonts.
 - `DM Serif Display` (Headings)
 - `DM Sans` (Body text)
 - `Outfit` (UI/Numbers/Monospaced data)

2. Core Design System & Theme Engine (Inject into

styles.css)

This CSS Variables matrix is the absolute core of the website's look. It dictates the sophisticated dark/light modes and the Liquid Glass effect. You must include these exact definitions and root resets.

css

```
:root {  
  /* Brand Palette Core */  
  --primary-dark: #0D2B14;  
  --primary-green: #13361F;  
  --accent-yellow: #F5E04A;  
  --accent-orange: #F2A65A;  
  /* Theme Defaults (Light Layout override) */  
  --bg-page: #F7F8F4;  
  --bg-main: #FFFFFF;  
  --bg-section: #FFFFFF;  
  --bg-section-alt: #EDEEE9;  
  --text-main: #1A1A1A;  
  --text-muted: #5C6B5E;  
  --border-color: rgba(0, 0, 0, 0.1);  
  /* The Premium Liquid Glass System (Critical aesthetic requirement) */  
  --glass-bg: rgba(255, 255, 255, 0.05); /* Extremely subtle frosting */  
  --glass-border: rgba(255, 255, 255, 0.08);  
  /* Typography Scale */  
  --font-heading: 'DM Serif Display', Georgia, serif;  
  --font-body: 'DM Sans', system-ui, sans-serif;  
  --font-ui: 'Outfit', system-ui, sans-serif;  
  /* Shared Geometry */  
  --radius-lg: 24px;  
  --radius-md: 16px;  
  --radius-pill: 100px;
```

```

}

/* Intelligent Dark Mode Overrides */
[data-theme="dark"] {
  --bg-page: #050505;
  --bg-main: #0A0A0A;
  --bg-section: #111111;
  --bg-section-alt: #0D0D0D;
  --text-main: #FFFFFF;
  --text-muted: #A0A0A0;
  --border-color: rgba(255, 255, 255, 0.1);
  --glass-bg: rgba(0, 0, 0, 0.6); /* Thicker frost on dark background */
  --glass-border: rgba(255, 255, 255, 0.1);
}

/* Global Setup & Layout Resets */
* { box-sizing: border-box; }
html { scroll-behavior: smooth; font-size: 16px; }
body {
  padding: 0;
  margin: 0;
  font-family: var(--font-body);
  color: var(--text-main);
  background: var(--bg-page);
  overflow-x: hidden;
}

/* Premium Liquid Glass Button Ecosystem */
.btn {
  display: inline-flex; align-items: center; justify-content: center; gap: 0.5rem;
  padding: 0.8rem 1.5rem; border-radius: var(--radius-pill);
  font-family: var(--font-ui); font-weight: 600; font-size: 0.95rem;

  /* The Frosted Effect */
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  text-decoration: none; cursor: pointer; transition: all 0.4s cubic-bezier(0.16, 1, 0.3, 1);
}

.btn-primary {

```

```

background: var(--accent-yellow); color: var(--primary-dark);
border: 1px solid rgba(245, 224, 74, 0.5);
box-shadow: 0 4px 10px rgba(0,0,0,0.05);
}

.btn-primary:hover {
transform: translateY(-3px);
box-shadow: 0 10px 30px rgba(245, 224, 74, 0.4);
}

.btn-ghost {
background: var(--glass-bg);
border: 1px solid var(--glass-border);
color: var(--text-main);
}

.btn-ghost:hover {
background: rgba(255,255,255,0.1);
}

```

3. High-Fidelity DOM Structure (Inject into

index.html)

You are to lay out the DOM sequentially exactly like this.

A. The Floating 'Liquid Glass' Navbar

This must be the primary navigation. It floats above all content as a frosted, translucent pill container, never as a massive solid block.

html

```
<!-- Secondary Navigation -->
```

```
<nav class="secondary-nav" style="position: fixed; top: 1rem; width: 100%; z-index: 9999; transform: translateY(-150%); transition: transform 0.4s cubic-bezier(0.16, 1, 0.3, 1); background: transparent; pointer-events: none; display: flex; justify-content: center; padding: 0 2rem;">
```

```
<!-- The actual floating 'pill' container -->
```

```

<div class="secondary-nav-inner" style="width: 100%; max-width: 1200px; padding: 0.75rem 1.5rem;
display: flex; align-items: center; justify-content: space-between; border-radius: 100px; background:
var(--glass-bg); backdrop-filter: blur(24px); -webkit-backdrop-filter: blur(24px); border: 1px solid
var(--glass-border); box-shadow: 0 16px 40px rgba(0, 0, 0, 0.08); pointer-events: auto;">

  <a href="/" class="sec-nav-logo" style="display: flex; align-items: center; gap: 0.5rem;
color: var(--text-main); font-family: var(--font-ui); font-size: 0.85rem; font-weight: 600;
text-transform: uppercase; text-decoration: none;">

    <div style="background: var(--accent-yellow); color: var(--bg-main); width: 24px; height: 24px;
border-radius: 50%; display: flex; align-items: center; justify-content: center; font-weight: bold;">☀</div>

    <span>photovoltaic goldstein</span>

  </a>

  <div class="sec-nav-links" style="display: flex; gap: 2.5rem;">

    <a href="/about" style="color: var(--text-muted); text-decoration: none; font-size: 0.95rem;
font-weight: 500;">About</a>

    <a href="/products" style="color: var(--text-muted); text-decoration: none; font-size: 0.95rem;
font-weight: 500;">Products</a>

    <a href="/solutions" style="color: var(--text-muted); text-decoration: none; font-size: 0.95rem;
font-weight: 500;">Solutions</a>

    <a href="/shop" style="color: var(--accent-orange); text-decoration: none; font-size: 0.95rem;
font-weight: 500;">Shop</a>

  </div>

  <div style="display: flex; gap: 1rem; align-items: center;">

    <a href="#cart" class="cart-icon" onclick="document.getElementById('main-cart').showModal();"
style="color: var(--text-main); cursor: pointer;"><i data-lucide="shopping-cart"></i></a>

    <a href="#contact" class="btn btn-primary">Contact Sales</a>

  </div>

</div>
</nav>

```

B. The Full-Bleed Video Hero Section

This sets the premium tone immediately.

```

html

<section class="hero-section" style="height: 100vh; overflow: hidden; position: relative; background:
#000; padding: 1rem;">

  <!-- Absolute background video -->
  <video autoplay muted loop playsinline style="position: absolute; top:0; left:0; width:100%; height:100%;
object-fit: cover; z-index: 1; opacity: 0.7;"></video>

  <!-- The inset glass frame container -->
  <div class="hero-glass-frame" style="position: relative; z-index: 2; width: 100%; height: 100%;
border-radius: 20px; box-shadow: inset 0 0 0 1px rgba(255,255,255,0.08); display: flex; flex-direction:
column; justify-content: flex-end; padding: 4rem;">

    <div style="display: flex; justify-content: space-between; align-items: flex-end;">
      <div style="max-width: 60%">
        <h1 style="font-family: var(--font-heading); color: white; font-size: clamp(3.5rem, 8vw, 6rem);
margin: 0 0 1.5rem 0; line-height: 0.95;">Power the future,<br/>elegantly designed.</h1>
        <a href="#shop" class="btn btn-primary" style="font-size: 1.1rem; padding: 1rem 2rem;">Start
Your Journey</a>
      </div>

      <div class="hero-info-widget" style="background: rgba(255,255,255,0.05); backdrop-filter:
blur(28px); -webkit-backdrop-filter: blur(28px); border-radius: 100px; padding: 0.75rem 1.5rem; display:
flex; align-items: center; border: 1px solid rgba(255,255,255,0.1);">
        <p style="color: white; margin: 0; font-family: var(--font-ui); font-size: 1rem;"><strong
style="color: var(--accent-yellow);">2.3M+</strong> kWh Generated</p>
      </div>
    </div>
  </div>
</section>

```

C. The Hardware Exploded View (Heavy Vanilla Canvas AI)

This section proves the "Engineering" claim. You must attach the script to this exact Canvas ID.

```
html
```

```

<section class="inverter-showcase" style="padding: 10rem 5%; background: var(--bg-main); display: grid;
grid-template-columns: 1fr 1fr; gap: 4rem; align-items: center;">

  <div class="canvas-container" style="position: relative; width: 100%; max-width: 800px; aspect-ratio:
1/1; margin: 0 auto;">

    <!-- The animation target -->

    <canvas id="explode-canvas" width="800" height="800" style="width: 100%; height: 100%; object-fit:
cover;"></canvas>

  </div>

  <div class="showcase-content">

    <h2 style="font-family: var(--font-heading); font-size: 3.5rem; margin-bottom: 1.5rem; color:
var(--text-main); line-height: 1;">Built to Outperform.</h2>

    <p style="color: var(--text-muted); font-size: 1.25rem; line-height: 1.6; margin-bottom: 2.5rem;
max-width: 90%;">Every component is engineered in Germany with micro-precision. The core heat
retention algorithms are mathematically unparalleled, storing excess energy across vast thermal
batteries.</p>

    <a href="/products" class="btn btn-ghost" style="padding: 1rem 2rem;">Explore Specifications</a>

  </div>
</section>

```

4. The Live E-Commerce Shopify Engine

You must use Shopify Storefront Web Components specifically as outlined below to create a direct purchase flow that connects actual inventory to the vanilla site.

1. Place the Shopify script loader right before the section:
2. html
3. `<script type="module" src="https://cdn.shopify.com/storefront/web-components.js"></script>`
4. Construct the highly styled 4-Column dynamic grid. Do not alter the specific product handles provided, as they map to live staging data.

```

html

<section class="shop-section" id="shop" style="padding: 8rem 5%; background: var(--bg-section-alt);">

  <div style="text-align: center; margin-bottom: 5rem;">

    <h2 style="font-family: var(--font-heading); font-size: 3.5rem; color: var(--text-main);">Top-Selling
Components</h2>

```

```

</div>

<!-- The Global Store Context -->
<shopify-store store-domain="rb6ppp-6r.myshopify.com"
public-access-token="d2f0468c56352badd67b92e3aa08333" country="US" language="en">

  <!-- Strict 4 Product Grid Container -->
  <div class="collection-grid" style="display: grid; grid-template-columns: repeat(4, 1fr); gap: 2rem;">

    <!-- PRODUCT 1: FoxESS -->
    <shopify-context type="product" handle="foxess-h3-8-0-e-hybrid-8kw">
      <template>
        <div shopify-attr--disabled="!product.availableForSale" class="product-card"
style="background: var(--bg-section); border-radius: 20px; padding: 1.5rem; box-shadow: 0 4px 20px
rgba(0,0,0,0.02); transition: transform 0.3s cubic-bezier(0.16, 1, 0.3, 1);">

          <!-- Image Layout with Modal Link -->
          <div class="product-card__image-container" style="position: relative; overflow: hidden;
border-radius: 12px; margin-bottom: 1.5rem; background: #fff; aspect-ratio: 4/3;">
            <a class="product__link"
onclick="document.getElementById('product-modal').showModal();
document.getElementById('product-modal-context').update(event);" style="cursor: pointer; display: block;
height: 100%;">
              <shopify-media max-images="1" width="400" height="300"
query="product.selectedOrFirstAvailableVariant.image" style="width: 100%; height: 100%; object-fit:
contain; padding: 1rem;"></shopify-media>
            </a>

            <!-- Liquid Glass hover button -->
            <button class="product-card__hover-button"
onclick="document.getElementById('main-cart').addLine(event).showModal();" style="position: absolute;
bottom: 15px; left: 50%; transform: translateX(-50%) translateY(100px); opacity: 0; background:
var(--glass-bg); backdrop-filter: blur(12px); color: var(--text-main); border: 1px solid var(--glass-border);
padding: 0.6rem 1.25rem; border-radius: 100px; font-family: var(--font-ui); font-weight: 700; font-size:
0.85rem; cursor: pointer; transition: all 0.3s cubic-bezier(0.16, 1, 0.3, 1);">
              Quick Add
            </button>
          </div>
        </div>
      </template>
    </shopify-context>
  </div>

```

```

        </button>
    </div>

    <!-- Details & Cart Logic -->
    <div class="product-card__details">
        <p style="color: var(--text-muted); font-size: 0.75rem; text-transform: uppercase;
letter-spacing: 0.1em; font-weight: 700; margin: 0 0 0.5rem 0;"><shopify-data
query="product.vendor"></shopify-data></p>
        <h3 style="font-family: var(--font-heading); color: var(--text-main); font-size: 1.4rem;
margin: 0 0 1.5rem 0; line-height: 1.2;"><shopify-data query="product.title"></shopify-data></h3>

        <div style="display: flex; justify-content: space-between; align-items: center; border-top:
1px solid var(--border-color); padding-top: 1rem;">
            <p style="font-family: var(--font-ui); font-weight: 700; font-size: 1.3rem; margin: 0;
color: var(--text-main);"><shopify-money
query="product.selectedOrFirstAvailableVariant.price"></shopify-money></p>
            <button onclick="document.getElementById('main-cart').addLine(event).showModal();"
style="background: var(--accent-yellow); color: var(--primary-dark); border: none; border-radius: 4px;
padding: 0.5rem 1rem; font-family: var(--font-ui); font-weight: 700; cursor: pointer;">Add to Cart</button>
        </div>
    </div>
</div>
</template>
</shopify-context>
<!-- PRODUCT 2: Deye SUN -->
<shopify-context type="product"
handle="deye-sun-20k-sg05lp3-eu-sm2-hybrid-wechselrichter-20-kw">
    <!-- USE IDENTICAL HTML TEMPLATE STRUCTRE FROM ABOVE -->
    <template><p>Implementation mirrors component 1.</p></template>
</shopify-context>

<!-- PRODUCT 3: Enameled Tank -->
<shopify-context type="product" handle="enameled-tank-oem-ceramic-202-g">
    <template><p>Implementation mirrors component 1.</p></template>
</shopify-context>

```

```

<!-- PRODUCT 4: Haier Super Aqua -->
<shopify-context type="product" handle="haier-super-aqua-11-kw">
  <template><p>Implementation mirrors component 1.</p></template>
</shopify-context>

</div>

<!-- Crucial Modals (Do not omit) -->
<!-- The Floating Cart side-drawer wrapper -->
<shopify-cart id="main-cart"></shopify-cart>

<!-- The Product Detailed High-Res Detail Page -->
<dialog id="product-modal" style="border:none; border-radius: 24px; padding:0; background:
transparent; max-width: 900px; width: 90vw;">
  <shopify-context id="product-modal-context" type="product" wait-for-update>
    <template>
      <div style="background: var(--bg-main); display: grid; grid-template-columns: 1fr 1fr;
border-radius: 24px; overflow: hidden; position: relative;">
        <!-- Close button -->
        <button onclick="document.getElementById('product-modal').close();" style="position:
absolute; top: 1rem; right: 1rem; background: var(--bg-section-alt); border: none; width: 40px; height:
40px; border-radius: 50%; font-size: 1.2rem; cursor: pointer; z-index: 10;">&times;</button>

        <div style="background: #fff; padding: 2rem;">
          <shopify-media max-images="1" width="500" height="500"
query="product.selectedOrFirstAvailableVariant.image" style="width: 100%; height: 100%; object-fit:
contain;"></shopify-media>
        </div>

        <div style="padding: 3rem; background: var(--bg-section);">
          <p style="color: var(--text-muted); text-transform: uppercase;"><shopify-data
query="product.vendor"></shopify-data></p>
          <h2 style="font-family: var(--font-heading); font-size: 2.2rem; color: var(--text-main);
margin-bottom: 0.5rem;"><shopify-data query="product.title"></shopify-data></h2>

```

```

    <p style="font-size: 1.5rem; font-weight: bold; margin-bottom: 2rem;"><shopify-money
query="product.selectedOrFirstAvailableVariant.price"></shopify-money></p>

    <div style="color: var(--text-muted); line-height: 1.6; margin-bottom: 2rem;">
        <shopify-data query="product.descriptionHtml"></shopify-data>
    </div>

    <shopify-variant-selector></shopify-variant-selector>

    <button onclick="document.getElementById('main-cart').addLine(event).showModal();
document.getElementById('product-modal').close();"
shopify-attr--disabled="!product.selectedOrFirstAvailableVariant.availableForSale" style="width: 100%;
background: var(--text-main); color: var(--bg-main); padding: 1rem; border: none; border-radius: 100px;
font-weight: bold; font-size: 1.1rem; cursor: pointer; margin-top: 1.5rem;">
        Add to Order
    </button>
</div>
</div>
</template>
</shopify-context>
</dialog>

</shopify-store>
</section>

```

5. The Core Operations JavaScript Algorithm (Inject into main.js)

This strict, untruncated javascript loop handles the math required for the floating navbar transformation and the heavy hardware animation drawing.

javascript

```
document.addEventListener("DOMContentLoaded", () => {
```

// 1. LIQUID GLASS FLOATING NAVBAR LOGIC

```
const secondaryNav = document.querySelector('.secondary-nav');
```

```
// Ensure Lucide icons render
```

```
if(typeof lucide !== 'undefined') lucide.createIcons();
```

```
if(secondaryNav) {
```

```
  window.addEventListener('scroll', () => {
```

```
    // Trigger 150px down the page
```

```
    if (window.scrollY > 150) {
```

```
      // Fly in the frosted pill
```

```
      secondaryNav.style.transform = "translateY(0)";
```

```
    } else {
```

```
      // Hide it aggressively
```

```
      secondaryNav.style.transform = "translateY(-150%)";
```

```
    }
```

```
  });
```

```
}
```

// 2. CONTINUOUS 30FPS HARDWARE CANVAS RENDER ENGINE

```
// This script drives the 3D exploding inverter effect cleanly without React.
```

```
const canvas = document.getElementById('explode-canvas');
```

```
if(canvas) {
```

```
  const ctx = canvas.getContext('2d');
```

```
  const frameCount = 240; // Total JPG frames available
```

```
  const currentFrame = index => `assets/frames/ezgif-frame-${(index + 1).toString().padStart(3, '0')}.jpg`;
```

```
  const images = [];
```

```
  // Asynchronous Pre-loader Block
```

```
  let loadedImages = 0;
```

```
  for (let i = 0; i < frameCount; i++) {
```

```
    const img = new Image();
```

```
    img.src = currentFrame(i);
```

```
    img.onload = () => { loadedImages++; };
```

```
    images.push(img);
```

```

    }

    let playhead = { frame: 0 };
    let lastTime = 0;

    // Target framerate throttle mechanism
    const fps = 30;
    const interval = 1000 / fps;
    function renderFrame(time) {
        requestAnimationFrame(renderFrame);

        // Frame throttle mathematics
        if (time - lastTime < interval) return;
        lastTime = time;
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        let img = images[Math.floor(playhead.frame)];

        if (img.complete) {
            // Complex mathematical approach to guarantee 'object-fit: cover' logic via Canvas API
            const scale = Math.max(canvas.width / img.width, canvas.height / img.height);
            const x = (canvas.width / 2) - (img.width / 2) * scale;
            const y = (canvas.height / 2) - (img.height / 2) * scale;
            ctx.drawImage(img, x, y, img.width * scale, img.height * scale);
        }

        // Loop infinitely back to frame 0
        playhead.frame = (playhead.frame + 1) % frameCount;
    }

    // Kickoff the loop immediately once the critical first frame has loaded
    images[0].onload = () => requestAnimationFrame(renderFrame);
}
});

```

Action Summary for the Target LLM:

You possess the architectural DNA. Combine the
 styles.css overrides with the

index.html DOM exactly as instructed, insert the
main.js algorithms without truncation, and compile the entire site cleanly. Do not skip any CSS property or
Shopify component logic. Execute immediately to produce the full site.

Shopifllm.md

Enhanced Shopify Storefront Web Components Rules

This file includes examples, guidelines, and documentation for building Storefront Web Components.

Purpose

Create a beautiful, production-ready HTML storefront using Shopify's Storefront Web Components based on the user's specific requirements.

Core Requirements

- Develop a responsive storefront using ONLY Shopify web components
- Create a streamlined shopping experience with proper product display and cart functionality
- Implement clean, minimal, production-ready code optimized for educational purposes

Design requirements

Strictly follow WCAG color contrast guidelines:

- All text and icons must have a minimum contrast ratio of 4.5:1 against their backgrounds (3:1 for large text/icons).
- UI components (buttons, input fields, icons) must have at least a 3:1 contrast ratio with adjacent colors.
- Use dark text/icons on light backgrounds or vice versa; avoid low-contrast pairs (e.g., light gray on white).
- Validate all color choices with a contrast checker and ensure compliance.

Maintain consistent and adequate whitespace between all elements:

- Use a spacing scale (e.g., multiples of 8px) for margins and paddings.
- Apply the law of proximity: group related elements together and separate unrelated elements with more space.
- Add generous padding around buttons and interactive elements for easy touch/click.
- Use a grid system for alignment and spacing consistency.
- Increase whitespace around headings, CTAs, and key content to create visual hierarchy.
- Ensure spacing adapts responsively across different screen sizes.

Maintain consistent typography throughout the designs

Product details modals must be aligned in the center of the page.

Display full images on product cards; avoid cropping images to fit.

Implementation Specifications

Required Components

- ****Store Configuration****: Properly configured `` element
- ****Product Display****: Complete product cards with images, titles, prices, and variants
- ****Cart Functionality****:
 - Working "Add to Cart" buttons and "Open Cart" button
 - Adding to cart should open the cart as well
- ****Responsive Design****: Layout that works across all device sizes

Technical Implementation

```
```html
<!-- Basic Structure (Expand Upon This) -->
<script
 type="module"
 src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store
 store-domain="mock.shop"
 country="US"
 language="en"
>

 <!-- Cart Component -->
 <shopify-cart id="main-cart"></shopify-cart>
 <button onclick="document.getElementById('main-cart').showModal()">
 Open Cart
 </button>

 <!-- Product Grid -->
 <div style="display: flex; flex-wrap: wrap; gap: 20px;">
 <shopify-context type="product" handle="hoodie">
 <template>
 <!-- Product card with image, title, price, variants, add-to-cart -->
 </template>
 </shopify-context>
 </div>
</shopify-store>
```
```

```

</div>
</shopify-store>
...

### Fully-functional example of embedded product card

<script type="module"
src="https://cdn.shopify.com/storefront/web-components.js"></script>
<!-- Provide the token and set the market context -->
<shopify-store store-domain="https://www.velasca.com" country="US"
language="en"></shopify-store>

<div class="product-layout">
  <div class="product-card">
    <!-- Set product you want to display -->
    <shopify-context type="product" handle="brumista-tdm">
      <template>
        <div class="product-card__container">
          <div class="product-card__media">
            <div class="product-card__main-image">
              <shopify-media width="280" height="280"
query="product.selectedOrFirstAvailableVariant.image"></shopify-media>
            </div>
          </div>
          <div class="product-card__details">
            <div class="product-card__info">
              <h2 class="product-card__title">
                <shopify-data query="product.title"></shopify-data>
              </h2>
              <div class="product-card__price">
                <shopify-money
query="product.selectedOrFirstAvailableVariant.price"></shopify-money>
              </div>
            </div>
            <button
              class="product-card__view-button"
              onclick="getElementById('product-modal').showModal();
getElementById('product-modal-context').update(event);"
              >
              View product
            </button>
          </div>
        </div>
      </template>
    </shopify-context>
  </div>
</div>

```

```

        </div>
    </div>
</template>
</shopify-context>
</div>
</div>

<shopify-cart id="cart"></shopify-cart>

<dialog id="product-modal" class="product-modal">
    <!-- The handle of this context is automatically set when the dialog is opened -->
    <shopify-context id="product-modal-context" type="product" wait-for-update>
        <template>
            <div class="product-modal__container">
                <div class="product-modal__close-container">
                    <button class="product-modal__close"
onclick="getElementById('product-modal').close();">&#10005;</button>
                </div>
                <div class="product-modal__content">
                    <div class="product-modal__layout">
                        <div class="product-modal__media">
                            <shopify-media width="416" height="416"
query="product.selectedOrFirstAvailableVariant.image"></shopify-media>
                        </div>
                        <div class="product-modal__details">
                            <div class="product-modal__header">
                                <div>
                                    <span class="product-modal__vendor">
                                        <shopify-data query="product.vendor"></shopify-data>
                                    </span>
                                </div>
                                <h1 class="product-modal__title">
                                    <shopify-data query="product.title"></shopify-data>
                                </h1>
                                <div class="product-modal__price-container">
                                    <shopify-money
query="product.selectedOrFirstAvailableVariant.price"></shopify-money>
                                </div>
                            </div>
                            <shopify-variant-selector></shopify-variant-selector>

                            <div class="product-modal__buttons">

```

```
<button
    class="product-modal__add-button"
    onclick="getElementById('cart').addLine(event).showModal();"
shopify-attr--disabled="!product.selectedOrFirstAvailableVariant.availableForSale"
>
    Add to cart
</button>
<button
    class="product-modal__buy-button"
    onclick="document.querySelector('shopify-store').buyNow(event)"
shopify-attr--disabled="!product.selectedOrFirstAvailableVariant.availableForSale"
>
    Buy now
</button>
</div>
<div class="product-modal__description">
    <span class="product-modal__description-text">
        <shopify-data query="product.descriptionHtml"></shopify-data>
    </span>
</div>
</div>
</div>
</div>
</div>
</template>
</shopify-context>
</dialog>

<style>
    @import
url('https://fonts.googleapis.com/css2?family=Inter:ital,opsz,wght@0,14..32,100..900;1
,14..32,100..900&display=swap');
    body {
        font-family: 'Inter', sans-serif;
        font-weight: 400;
        font-style: normal;
    }
    /** Product card style */
    .product-layout {
        display: flex;
```

```
}

.product-card {
  max-width: 80rem;
  background-color: #ffffff;
  display: flex;
  flex-direction: column;
  gap: 1em;
  border-radius: 12px;
}

.product-card__container {
  display: flex;
  flex-direction: column;
  padding: 1rem;
  box-sizing: border-box;
}

.product-card__media {
  display: flex;
  flex-direction: column;
  align-items: center;
  width: 100%;
}

.product-card__main-image img {
  border-radius: 0.5rem;
}

.product-card__details {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
  width: 100%;
  margin-top: 0.5rem;
}

.product-card__info {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.product-card__title {
  font-size: 1.125rem;
  font-weight: 700;
  margin: 0;
}

.product-card__price {
```

```
display: flex;
gap: 0.5rem;
font-size: 1rem;
font-weight: 500;
}
.product-card__view-button {
background-color: #000000;
color: #ffffff;
border-radius: 0.75rem;
padding: 1rem;
font-size: 0.875rem;
font-weight: 700;
text-transform: uppercase;
transition: background-color 0.3s ease, color 0.3s ease;
border: 2px solid black;
cursor: pointer;
}
.product-card__view-button:hover {
background: #ffffff;
color: #000000;
border: 2px solid black;
}
/** Modal styles **/
.product-modal {
padding: 0;
border-radius: 0.75rem;
box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.25);
border: 0;
}
.product-modal::backdrop {
background-color: rgba(156, 163, 175, 0.5);
}
.product-modal__container {
position: relative;
overflow-x: hidden;
padding: 2rem;
}
.product-modal__close-container {
display: grid;
justify-items: end;
justify-content: end;
margin-left: 2rem;
```

```
padding-bottom: 1rem;
}
.product-modal__close {
border-radius: 12px;
width: 32px;
height: 32px;
border: 0;
cursor: pointer;
}
.product-modal__content {
width: 100%;
background-color: #ffffff;
border-radius: 0.75rem;
max-width: 54rem;
min-width: 320px;
}
.product-modal__layout {
display: flex;
flex-direction: column;
gap: 2rem;
}
@media (min-width: 768px) {
.product-modal__layout {
flex-direction: row;
}
}
.product-modal__media {
display: flex;
flex-direction: column;
align-items: center;
width: 100%;
}
.product-modal__media img {
border-radius: 0.25rem;
width: 100%;
}
.product-modal__details {
display: flex;
flex-direction: column;
gap: 2rem;
width: 100%;
}
```

```
.product-modal__header {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
}

.product-modal__vendor {
  opacity: 0.5;
  font-weight: 700;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  font-size: 0.75rem;
}

.product-modal__title {
  font-size: 2.25rem;
  font-weight: 700;
  margin: 0;
}

.product-modal__price-container {
  display: flex;
  gap: 0.5rem;
  font-weight: 500;
  font-size: 1.25rem;
}

.product-modal__compare-price {
  text-decoration: line-through;
  opacity: 0.5;
}

.product-modal__buttons {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.product-modal__buttons button {
  font-size: 0.875rem;
  font-weight: 800;
  transition: background-color 0.3s ease, color 0.3s ease;
  cursor: pointer;
  border: 2px solid black;
  text-transform: uppercase;
}

.product-modal__add-button {
  background-color: #000000;
```

```
    color: #ffffff;
    border-radius: 0.75rem;
    padding: 1rem;
    font-size: 18px;
    font-weight: 500;
    width: 100%;
  }
  .product-modal__add-button:hover {
    background: #ffffff;
    color: #000000;
    border: 2px solid black;
  }
  .product-modal__add-button:disabled {
    opacity: 0.3;
  }
  .product-modal__buy-button {
    background-color: #ffffff;
    color: #000000;
    border-radius: 0.75rem;
    padding: 1rem;
    font-size: 18px;
    font-weight: 500;
    width: 100%;
  }
  .product-modal__buy-button:disabled {
    opacity: 0.3;
  }
  .product-modal__buy-button:hover {
    opacity: 0.3;
  }
  .product-modal__description-text {
    font-weight: 400;
    color: #717171;
    letter-spacing: 0.05em;
    font-size: 0.875rem;
  }
  .product-modal__description-text p {
    margin: 0;
  }
</style>
```

Technical Tips

- For cart modal after adding items:
``document.getElementById('main-cart').showModal()``
- In the ``shopify-cart::part(secondary-button)`` CSS block, always include a ``fill`` property.
- For context updating in dialogs:
``document.getElementById('context-id').updateContext(event)``
- Default product handles if none specified: ``hoodie``, ``sweatpants``, ``puffer``
- For collection pages, use this handle if none specified: ``unisex``
- Include error handling for product variants and loading states
- For grid css columns - don't use ``auto-fill``
- Don't add attributes on shopify elements that are not specified in the documentation
- For product image, using ``product.selectedOrFirstAvailableVariant.image`` is better than ``product.featuredImage`` if you want it to change when a variant is selected
- Always include this script tag: `<script type="module" src="https://cdn.shopify.com/storefront/web-components.js"></script>`

Example Steps Breakdown Ideas (if the user asks for something like this)

1. A Minimal Product Card with Image and Price

Implementation Steps:

1. Set up the store configuration using `<shopify-store>` with a ``store-domain`` attribute
2. Create a product context using `<shopify-context type="product" handle="product-handle">`
3. Inside the template:
 - Add a `<shopify-media>` component to display the product image
 - Add a heading with `<shopify-data>` to show the product title
 - Add `<shopify-money>` to display the price
4. Apply basic styling to create a card-like appearance

2. A Cool Example Using the 'Buy Now' Button

Implementation Steps:

1. Set up store configuration with `<shopify-store>`
2. Create a product context
3. Inside the template:
 - Add product details (title, description) using `<shopify-data>`

- Add variant selector with ``<shopify-variant-selector>``
 - Create a "Buy Now" button that references the cart component
 - Set up a ``<shopify-cart>`` component with the buyNow method
4. Configure the button's onclick event to use the cart's buyNow method
 5. Style the button for visual appeal

3. A Grid Collection Page

Implementation Steps:

1. Set up store configuration
2. Create a collection context: ``<shopify-context type="collection" handle="collection-handle">``
3. Display collection title and description
4. Add a nested product context for the collection's products:
``<shopify-context type="product" query="collection.products" first="12">``
5. Create a grid layout using CSS Grid or Flexbox
6. Within the product template, create product cards with:
 - Product image using ``<shopify-media>``
 - Product title using ``<shopify-data>``
 - Product price using ``<shopify-money>``
7. Add pagination or "load more" functionality if needed

4. A Product Details Page Layout

Implementation Steps:

1. Set up store configuration
2. Create a product context
3. Design a two-column layout:
 - Left column:
 - Product images gallery using ``<shopify-media>`` for the featured image
 - Additional image handling if needed
 - Right column:
 - Product title using ``<shopify-data query="product.title">``
 - Product description using ``<shopify-data query="product.description">``
 - Price using ``<shopify-money>``
 - Variant selector using ``<shopify-variant-selector>``
 - Add to cart button that calls a cart component
4. Add a ``<shopify-cart>`` component for cart functionality

5. Style with responsive design in mind

5. A Blog Post Page Layout

****Implementation Steps:****

1. Set up store configuration
2. Create a blog article context:
```  
  
<shopify-context type="article" handle="article-handle">  
```
3. Design the blog post layout:
 - Header section with:
 - Article title using ``<shopify-data query="article.title">``
 - Published date using ``<shopify-data query="article.publishedAt">``
 - Author information if available
 - Featured image section using ``<shopify-media>``
 - Content section using ``<shopify-data query="article.content">``
 - Tags or categories section if needed
4. Add navigation for related articles or back to blog link
5. Style appropriately for readability

6. A Very Customized Variant Selector

****Implementation Steps:****

1. Set up store configuration
2. Create a product context
3. Add a basic ``<shopify-variant-selector>`` component
4. Create a detailed CSS customization targeting parts:
```css  
  
shopify-variant-selector::part(form) {  
 /\* Custom layout \*/  
}  
  
shopify-variant-selector::part(label) {  
 /\* Custom label styling \*/  
}  
  
shopify-variant-selector::part(radio) {  
 /\* Custom radio button styling \*/  
}  
  
shopify-variant-selector::part(radio-selected) {  
 /\* Custom styling for selected options \*/  
}

```

}

shopify-variant-selector::part(color-swatch) {
 /* Custom color swatch styling */
}

shopify-variant-selector::part(color-swatch-selected) {
 /* Custom styling for selected color swatches */
}

...

```

5. Add JavaScript to enhance interactivity if needed

6. Ensure the variant selector updates product images and prices appropriately

## ## Response Format

- Return ONLY HTML code that goes between ``<body>`` and ``</body>`` tags
- Prioritize any specific requirements from the user's prompt
- No explanations or comments outside the HTML code
- Code should be production-ready with proper component structure

Remember: The user's specific requirements always take precedence over these general guidelines.

-----

## # Shopify Storefront Components API Documentation

### ## Table of Contents

- [Attributes] ([#attributes](#))
  - [shopify-attr] ([#shopify-attr](#))
- [Components] ([#components](#))
  - [shopify-cart] ([#shopify-cart](#))
  - [shopify-context] ([#shopify-context](#))
  - [shopify-list-context] ([#shopify-list-context](#))
  - [shopify-data] ([#shopify-data](#))
  - [shopify-media] ([#shopify-media](#))
  - [shopify-money] ([#shopify-money](#))
  - [shopify-store] ([#shopify-store](#))
  - [shopify-variant-selector] ([#shopify-variant-selector](#))

### ## Attributes

#### ### shopify-attr

**\*\*Description\*\*:** Use the ``shopify-attr`` attribute to bind an attribute to data from Shopify. Anywhere within the template of a `[shopify-context component]` ([#shopify-context](#)), you can use the ``shopify-attr--attribute-name`` attribute to bind an attribute to data from Shopify. For example, ``shopify-attr--href="product.onlineStoreUrl"`` can be used to bind the ``href`` attribute to the ``onlineStoreUrl`` field on a product context.

See the [\[playground\]](#) (<https://webcomponents.shopify.dev/playground>) for more complete examples.

**\*\*Example\*\*:**

```
```html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store
  store-domain="https://your-store.myshopify.com"
  country="CA"
  language="FR"
>
</shopify-store>
<shopify-context type="product" handle="your-product-handle">
  <template>
    <!-- the href attribute is bound to the
         product.onlineStoreUrl field -->
    <a shopify-attr--href="product.onlineStoreUrl"> View product </a>
  </template>
</shopify-context>
```
```

## ## Components

### ### shopify-cart

**\*\*Description\*\*:** The cart component provides a mini shopping cart functionality for your website. Here's how it works:

1. To add items to the cart:

- Use the ``addLine()`` method
- The method needs an event object
- The event's target must be inside a product [context component] ([#shopify-context](#))

2. To display the cart:

- The cart uses a native [HTML `<dialog>` element] (<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dialog>).
- To show it as a popup modal, call the ``showModal()`` method.

3. Customize the cart with CSS parts and slots.

> Note:

> The cart component does not support mixing products from multiple stores.

**\*\*Example\*\*:**

```

` ``html
<script
 type="module"
 src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
 <template>
 <shopify-variant-selector></shopify-variant-selector>
 <!-- The product added will be whatever
variant is selected for the context product handle -->
 <button onclick="getElementById('cart').addLine(event).showModal();">
 Add to cart
 </button>
 </template>
</shopify-context>

<shopify-cart id="cart"></shopify-cart>
` ``

```

#### Attributes and properties

| Name      | Type                         | Description                                                                                                                                                                                     |
|-----------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Optional  |                              |                                                                                                                                                                                                 |
| -----     | -----                        |                                                                                                                                                                                                 |
| -----     | -----                        |                                                                                                                                                                                                 |
| addLine   | (e: Event) => CartAttributes | A method to add an item to the cart.                                                                                                                                                            |
| Yes       |                              |                                                                                                                                                                                                 |
| close     | () => CartAttributes         | A method to close the cart dialog.                                                                                                                                                              |
| Yes       |                              |                                                                                                                                                                                                 |
| open      | boolean                      | A property to get the open state of the cart. Example: <code>getElementById('cart').open</code>                                                                                                 |
| Yes       |                              |                                                                                                                                                                                                 |
| show      | () => CartAttributes         | A method to display the cart as a modal in a <code>[`dialog` element</code>                                                                                                                     |
|           |                              | <code>modelessly]</code> ( <a href="https://developer.mozilla.org/en-US/docs/Web/API/HTMLDialogElement/show">https://developer.mozilla.org/en-US/docs/Web/API/HTMLDialogElement/show</a> ).     |
| Yes       |                              |                                                                                                                                                                                                 |
| showModal | () => CartAttributes         | A method to display the underlying <code>[cart</code>                                                                                                                                           |
|           |                              | <code>as a</code>                                                                                                                                                                               |
|           |                              | <code>modal]</code> ( <a href="https://developer.mozilla.org/en-US/docs/Web/API/HTMLDialogElement/showModal">https://developer.mozilla.org/en-US/docs/Web/API/HTMLDialogElement/showModal</a> ) |
|           |                              | in a <code>`dialog` element</code> .   Yes                                                                                                                                                      |
| target    | string                       | The <code>[target</code>                                                                                                                                                                        |
|           |                              | <code>attribute]</code> ( <a href="https://developer.mozilla.org/en-US/docs/Web/HTML/Element/a#target">https://developer.mozilla.org/en-US/docs/Web/HTML/Element/a#target</a> ) for the         |
|           |                              | checkout link. Defaults to <code>"\_top"</code> .   Yes                                                                                                                                         |

#### #### CSS parts

CSS parts allow you to target and override the default styling within the cart component.

| Part Name    | Description                         |
|--------------|-------------------------------------|
| -----        |                                     |
| -----        |                                     |
| dialog       | The dialog element.                 |
| line-heading | The cart line-item title element.   |
| line-image   | The cart line-item image element.   |
| line-options | The cart line-item options element. |

|                  |                                                                                       |
|------------------|---------------------------------------------------------------------------------------|
| line-price       | The cart line-item quantity element.                                                  |
|                  |                                                                                       |
| primary-button   | The primary button element. Used to style the checkout link.                          |
|                  |                                                                                       |
| secondary-button | The secondary button element. Used to style the buttons that modify a cart-line item. |

#### #### Slots

Slots allow you to override the default content of the cart component.

|                 |                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| Slot Name       | Description                                                                                                        |
|                 |                                                                                                                    |
| -----           |                                                                                                                    |
| -----           |                                                                                                                    |
| -----           |                                                                                                                    |
| checkout-button | The content to display in the checkout button. Useful to add a custom checkout button text.                        |
| empty           | The content to display when the cart is empty.                                                                     |
|                 |                                                                                                                    |
| extension       | Extend the cart with additional content below the checkout button. Useful to add upsell products or other content. |

#### \*\*Custom Cart Example\*\*:

```

```html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-cart>
  <!-- Override the empty state with translated text -->
  <div slot="empty">Ihr Warenkorb ist leer</div>
  <!-- Override the checkout button with translated text -->
  <div slot="checkout-button">Zur Kasse</div>
</shopify-cart>

<style>
shopify-cart::part(dialog) {
  border-radius: 0.5rem;
}
shopify-cart::part(primary-button) {

```

```

    background-color: #627059;
    border: 0;
    border-radius: 0;
    color: #ffffff;
    font-size: 0.875rem;
    font-weight: 500;
  }
  shopify-cart::part(secondary-button) {
    background-color: #ffffff;
    color: #000;
    fill: #000;
    border: 2px solid #000;
    border-radius: 0;
  }
</style>
` ``

```

shopify-context

****Description**:** The context component defines which Shopify data should be available in different parts of your page.

Each `` component requires two attributes:

- `type`: Specifies what kind of data you want (for example, `product`).
- `handle` or `gid`: Identifies the specific item. For example, the handle for the URL [`demostore.mock.shop/products/men-t-shirt`] (<https://demostore.mock.shop/products/men-t-shirt>) is `men-t-shirt`. The `gid` attribute can be used to identify the item by its unique id, e.g. `gid://shopify/Product/7982853619734`.

If you're working with a single storefront, then you can add the `` component anywhere on your page (it doesn't need to be inside the `` component). If you're working with multiple storefronts, then nest the context inside its corresponding store component.

Every `` component also requires a `` component, which contains the data you want to display. That template won't be rendered until the context is loaded. Render placeholder content outside the template with an attribute `shopify-loading-placeholder`. This content will be displayed until the context is loaded.

See the [playground](https://webcomponents.shopify.dev/playground) for complete examples.

****Related**:** [\[shopify-list-context\]](#) ([#shopify-list-context](#))

****Example**:**

```

`html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
  <template>
    <!-- The data component is bound to the product
    context and queries the title field -->
    <h1>
      <shopify-data query="product.title"></shopify-data>
    </h1>
  </template>
  <!-- Render placeholder content
  until the context is loaded -->
  <div shopify-loading-placeholder>Loading...</div>
</shopify-context>
`

```

Attributes

```
| Name | Type | Description  
| Optional |  
| ----- | ----- |  
  
-----  
  
-----  
  
-----  
  
-----  
  
--- | ----- |
```

| handle | string | The handle for the context. Required on some objects, like products, collections, and blogs. The handle is required unless the ``wait-for-update`` attribute is included.

| Yes |

| query | string | The query path for accessing nested data within a parent context. Required when this context is nested inside another context. The query should specify the path to access the desired data from the parent. Example: If the parent context is a product, and you want to access its first available variant: `query="product.selectedOrFirstAvailableVariant"`

| Yes |

| type | string | The type of the context. This needs to match the [GraphQL Storefront API](https://shopify.dev/docs/api/storefront) type you are querying. For example, if you are querying a product, the type should be ``type="product"``.

| No |

| update | (e: Event) => void | Updates this context to match the data from another context of the same type. Common use case: When displaying a list of products, you might want to show a detailed view of a single product in a modal. This method allows you to update the modal's context to display the selected product's data. How it works: 1. The event target must be inside the source context you want to copy from 2. This context will update its handle to match the source context 3. The data will be automatically refreshed to show the new content | Yes |

| wait-for-update | boolean | Wait to render the context until the update method is called. This is useful for dynamically rendering a context.

| Yes |

****Example - Updating a context with a dialog**:**

```

` ``html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store
  store-domain="https://your-store.myshopify.com"
  country="CA"
  language="FR"
>
</shopify-store>

<script>
  function showProductDetails(event) {

```

```

    // Update a dialog context with a selected product
    document.getElementById("dialog-context").update(event);

    // Show the dialog
    document.getElementById("dialog").showModal();
  }
</script>

<shopify-list-context type="product" query="products" first="10">
  <!-- This template is repeated for each product-->
  <template>
    <button onclick="showProductDetails(event)">
      <shopify-data query="product.title"></shopify-data>
    </button>
  </template>
</shopify-list-context>

<dialog id="dialog">
  <shopify-context id="dialog-context" type="product" wait-for-update>
    <template>
      <div>
        <shopify-data query="product.description"></shopify-data>
      </div>
    </template>
    <div shopify-loading-placeholder>Loading...</div>
  </shopify-context>
</dialog>
...

```

****Example - Paginated list of products**:**

```

```html
<script
 type="module"
 src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store
 store-domain="https://your-store.myshopify.com"
 country="CA"
 language="FR"
>
</shopify-store>

```

```

<shopify-list-context type="product" query="products" first="10">
 <!-- This template is repeated for each product-->
 <template>
 <shopify-data query="product.title"></shopify-data>
 </template>
</shopify-list-context>
<button id="previous" onclick="getElementById('list-context').previousPage();">
 Previous
</button>
<button id="next" onclick="getElementById('list-context').nextPage();">
 Next
</button>

```

```

<script>
 // Listen for the list context to update
 // and disable the next and previous buttons when
 // the list is at the end or beginning
 document
 .querySelector("shopify-context")
 .addEventListener("shopify-list-context-update", (event) => {
 const { hasNextPage, hasPreviousPage } = event.detail;
 if (!hasNextPage) {
 document.getElementById("next").setAttribute("disabled", "true");
 } else {
 document.getElementById("next").removeAttribute("disabled");
 }
 if (!hasPreviousPage) {
 document.getElementById("previous").setAttribute("disabled", "true");
 } else {
 document.getElementById("previous").removeAttribute("disabled");
 }
 });
</script>
...

```

### ### shopify-list-context

**\*\*Description\*\*:** The list context component allows you to display multiple items in a repeating format. To use it, you need three key attributes:

1. ``type``: Defines what you're listing (such as 'product' or 'collection')

2. ``query``: Specifies which data fields you want to display
3. ``first``: Sets the number of items to show in the list

Inside the list context, a template component defines how each item should appear. This template will automatically repeat for each item in your list. When you reference data within the template (using `shopify-data` or other components), it will automatically pull from the current item being displayed.

> Note:

> The list context can be nested inside a context component or other list context components.

See the [playground] (<https://webcomponents.shopify.dev/playground>) for examples.

**\*\*Related\*\*:** [`shopify-context`] ([#shopify-context](#))

**\*\*Example\*\*:**

```
```html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-list-context type="product" query="products" first="10">
  <!-- The template is repeated for each item
       in the array -->
  <template>
    <h2>
      <shopify-data query="product.title"></shopify-data>
    </h2>
  </template>
</shopify-list-context>
```
```

#### #### Attributes

| Name     | Type | Description |
|----------|------|-------------|
| Optional |      |             |

```

| ----- | ----- |
|
| ----- | ----- |
|
| first | number | The number of items to return.
| No |
| nextPage | () => void | Load the next page of items in the list.
| Yes |
| previousPage | () => void | Load the previous page of items in the list.
| Yes |
| query | string | Defines where the list exists, either at the root or
relative to a parent context. For example: 1. At the root, query a list of all
products, `query="products"` 2. Within a parent collection context, query the products
on that collection, `query="collection.products"` | No |
| reverse | () => void | Reverse the order of the items in the list.
| Yes |
| type | string | The type of the context. This needs to match the
[GraphQL Storefront API] (https://shopify.dev/docs/api/storefront) type you are
querying. For example, if you are querying a product, the type should be
`type="product"`. | No |

```

### ### shopify-data

**\*\*Description\*\*:** The shopify-data component is used to display Shopify data on your page. Here's how it works:

- It requires a `query` attribute that specifies which data to display.
- The query uses dot notation to access data fields (for example, `query="product.title"`).
- It looks for the nearest matching context to find the data.
- It outputs plain text that you can style with your own HTML elements.

For example:

`<shopify-data query="product.title">` will:

1. Find the nearest product context.
2. Access its title property.
3. Display the result as text.

Since the component outputs a text node, you can wrap it in any HTML elements to match your site's design. The component also supports rich text fields like ``product.descriptionHtml``.

See the [playground] (<https://webcomponents.shopify.dev/playground>) for more complete examples.

#### **\*\*Example\*\*:**

```
```html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
  <template>
    <h1>
      <!-- Query the title of the product.
           Renders a text node with display: contents -->
      <shopify-data query="product.title"></shopify-data>
    </h1>
  </template>
</shopify-context>
```
```

#### **#### Attributes**

| Name     | Type   | Description                                                                                                                                              |
|----------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Optional |        |                                                                                                                                                          |
| -----    | -----  |                                                                                                                                                          |
| -----    |        |                                                                                                                                                          |
| query    | string | Defines the context to reference and field to query. For example <code>`query="product.title"`</code> would query the title of the product context.   No |

#### **### shopify-media**

**\*\*Description\*\*:** Accepts a reference to an [Image] (<https://shopify.dev/docs/api/storefront/latest/objects/Image>) or

[Media] (<https://shopify.dev/docs/api/storefront/latest/interfaces/Media>) reference and generates an image or video element with `srcset` and `sizes` attributes. This component must be a child of a `shopify-context` component. It takes a query attribute that defines the context it's a part of, and the field to query.

If you want the media to automatically change based on which variant is selected on the [variant-selector component] ([#shopify-variant-selector](#)), make sure to reference the product image on the `product.selectedOrFirstAvailableVariant.image` field.

See the [playground] (<https://webcomponents.shopify.dev/playground>) for more complete examples.

> Note:

> When rendering an image, the media component uses the [`unpic-img`] (<https://unpic.pics/img/lit/>) element internally, so you can also pass `height`, `width`, `layout`, `aspect-ratio`, `priority`, `breakpoints`, and `sizes` attributes to control the scale and size of the image. Learn more about image props in the [Unpic documentation] (<https://unpic.pics/img/lit/#image-props>).

**\*\*Example\*\*:**

```
```html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
  <template>
    <h1>
      <!-- Query the featured image of the product.
           Renders an image element -->
      <shopify-media
        width="200"
        height="300"
        query="product.featuredImage"
      ></shopify-media>
    </h1>
  </template>
</shopify-context>
```

Attributes

Name	Type	Description
Optional		
-----	-----	-----
-----	-----	-----
-----	-----	-----
-----	-----	-----
aspectRatio	number	Instead of providing a width and height, you can provide an aspect ratio. This is passed to the <code>[aspectRatio]</code> (https://unpic.pics/img/webc/#aspect-ratio) attribute of an underlying <code>`unpic-img`</code> element.
No		
breakpoints	string	The breakpoints of the image. This is passed to the <code>[breakpoints]</code> (https://unpic.pics/img/webc/#breakpoints) attribute of an underlying <code>`unpic-img`</code> element.
Yes		
height	number	The height of the image. Required, unless width is provided with an aspectRatio.
No		
layout	"fixed" \ "constrained" \ "fullWidth"	The resizing behavior of the image. This is passed to the <code>[layout]</code> (https://unpic.pics/img/webc/#layout) attribute of an underlying <code>`unpic-img`</code> element.
Yes		
priority	boolean	Whether to prioritize the image. This is passed to the <code>[priority]</code> (https://unpic.pics/img/webc/#priority) attribute of an underlying <code>`unpic-img`</code> element.
Yes		
query	string	Defines the context to reference and field to query. For example, <code>`query="product.featuredImage"`</code> queries the title of the product featured image, and <code>`query="product.selectedOrFirstAvailableVariant.image"`</code> queries the image of a specific product variant based on the <code>`shopify-variant-selector`</code> component.
No		
role	string \ null	The accessibility role of the image. This is set automatically by the media component, but you can override it if needed.
Yes		

sizes	string	The sizes of the
-------	--------	------------------

image. This is set automatically by the media component, but you can override it if needed.

Yes		
-----	--	--

video-autoplay	boolean	Used for video media.
----------------	---------	-----------------------

By default, videos

[autoplay] (<https://developer.mozilla.org/docs/Web/HTML/Element/video#autoplay>). To disable autoplay, set to `video-autoplay="false"`.

Yes		
-----	--	--

video-controls	boolean	Used for video media.
----------------	---------	-----------------------

By default, [video

controls] (<https://developer.mozilla.org/docs/Web/HTML/Element/video#controls>) are shown. To disable them, set to `video-controls="false"`.

Yes		
-----	--	--

video-loop	boolean	Used for video media.
------------	---------	-----------------------

By default, videos

[loop] (<https://developer.mozilla.org//docs/Web/HTML/Element/video#loop>). To disable looping, set to `video-loop="false"`.

Yes		
-----	--	--

video-muted	boolean	Used for video media.
-------------	---------	-----------------------

By default, videos are

[muted] (<https://developer.mozilla.org/docs/Web/HTML/Element/video#muted>). To enable audio, set to `video-muted="false"`.

Yes		
-----	--	--

video-playsinline	boolean	Used for video media.
-------------------	---------	-----------------------

By default, videos [play

inline] (<https://developer.mozilla.org/docs/Web/HTML/Element/video#playsinline>). To disable inline playback, set to `video-playsinline="false"`.

Yes		
-----	--	--

width	number	The width of the
-------	--------	------------------

image. Required, unless height is provided with an aspectRatio.

No		
----	--	--

shopify-money

****Description**:** Accepts query a reference to a [Money object] (<https://shopify.dev/docs/api/storefront/2024-04/objects/MoneyV2>), and uses the store's country and language market to format it correctly. This component must be a child of a [`shopify-context`] (`#shopify-context`) component. The component takes a query attribute that defines the context it's a part of, and the field to query. This component produces a text node with the formatted price.

See the [playground] (<https://webcomponents.shopify.dev/playground>) for more complete examples.

```

` ``html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
  <template>
    <h1>
      <!-- Query the selected or first available variant's
price. Renders a text node with the formatted price. -->
      <shopify-money
        format="money_with_currency"
        query="product.selectedOrFirstAvailableVariant.price"
      ></shopify-money>
    </h1>
  </template>
</shopify-context>
` ``

```

[illegible]

| format | "money" \| "money_without_currency" \| "money_with_currency" | The format of the price. Defaults to `money`. Options: `money` - Display the price in the store's currency. eg. `\$100.00`; `money\_without\_currency` - Display the price in the store's currency, without the currency symbol. eg. `100.00`; `money\_with\_currency` - Display the price in the store's currency, including the currency symbol. eg. `\$100.00 USD` | Yes |

| query | string | Defines the context to reference and field to query. For example `query="product.title"` would query the title of the product context.

| No |

shopify-store

****Description**:** Use the `` component to set up your credentials and market configuration for a storefront. You can optionally add a public access token, which gives you access to inventory, metafields, and metaobjects. You can get a public access token by adding the [Headless channel] (</docs/storefronts/headless/building-with-the-storefront-api/manage-headless-channels>) in your Shopify admin.

See the [playground] (<https://webcomponents.shopify.dev/playground>) for more complete examples.

****Example**:**

```
```html
<script
 type="module"
 src="https://cdn.shopify.com/storefront/web-components.js"
></script>

<!-- Optionally define market configuration, which defaults
to US/EN. The public-access-token attribute is optional,
and only necessary to access inventory, metafields,
and metaobjects.
-->
<shopify-store
 store-domain="https://your-store.myshopify.com"
 public-access-token="optional-access-token"
 country="CA"
 language="FR"
>
</shopify-store>
```

```
<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
 <template> ... </template>
</shopify-context>
```

```
<!-- If you want to display products from multiple
storefronts on the same page, nest contexts inside
multiple store components-->
<shopify-store
 store-domain="https://your-other-store.myshopify.com"
 country="CA"
 language="FR"
>
 <shopify-context type="product" handle="handle-of-product">
 <template> ... </template>
 </shopify-context>
</shopify-store>
...

```

#### #### Attributes

Name	Type	Description
Optional		
-----	-----	-----
buyNow	(e: Event, target: Target) => void	A method to open the checkout page with a selected product.
Yes		
country	CountryCode	The country of the store.
Yes		
language	LanguageCode	The language of the store.
Yes		
public-access-token	string	The public access token from the <a href="/docs/storefronts/headless/building-with-the-storefront-api/manage-headless-channels">[Headless channel]</a> ( <a href="/docs/storefronts/headless/building-with-the-storefront-api/manage-headless-channels">/docs/storefronts/headless/building-with-the-storefront-api/manage-headless-channels</a> ) for the store.
Yes		
store-domain	string	The myshopify.com domain of the store.
No		

### ### shopify-variant-selector

**\*\*Description\*\*:** Use the `` component to display a form for selecting product options. The variant selector must be a child of a product context. Any data, money, or media component that references `selectedOrFirstAvailableVariant` will automatically update when a variant is selected.

See the [playground] (<https://webcomponents.shopify.dev/playground>) for more complete examples.

#### **\*\*Example\*\*:**

```
```html
<script
  type="module"
  src="https://cdn.shopify.com/storefront/web-components.js"
></script>
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>

<!-- The context is bound to the store -->
<shopify-context type="product" handle="handle-of-product">
  <template>
    <shopify-variant-selector></shopify-variant-selector>

    <!-- Fields on `selectedOrFirstAvailableVariant`
         automatically update when a variant is selected -->
    <shopify-money
      query="product.selectedOrFirstAvailableVariant.price"
    ></shopify-money>

    <shopify-media
      query="product.selectedOrFirstAvailableVariant.image"
      width="200"
      height="200"
    ></shopify-media>
  </template>
</shopify-context>
```
```

### #### Attributes



| radio-selected            | The radio selected element.

|

| select                    | The select element.

|

#### **\*\*Custom Variant Selector Example\*\*:**

```
```html
```

```
<script
```

```
  type="module"
```

```
  src="https://cdn.shopify.com/storefront/web-components.js"
```

```
></script>
```

```
<shopify-store store-domain="https://your-store.myshopify.com"> </shopify-store>
```

```
<!-- The context is bound to the store -->
```

```
<shopify-context type="product" handle="handle-of-product">
```

```
  <template>
```

```
    <shopify-variant-selector> </shopify-variant-selector>
```

```
  </template>
```

```
</shopify-context>
```

```
<style>
```

```
  shopify-variant-selector::part(form) {
```

```
    /** Change the layout of the variant selector */
```

```
    flex-direction: row;
```

```
    gap: 10px;
```

```
  }
```

```
  shopify-variant-selector::part(label) {
```

```
    /** Change the label of the variant selector */
```

```
    font-weight: bold;
```

```
  }
```

```
</style>
```

```
```
```

Agent\_sop.md

```
🚀 AGENT_SOP.md — Standard Operating Protocol for AI Agents
Applies to: Antigravity IDE · Gemini CLI · Claude Code · Cursor · Codex CLI ·
OpenCode · AdaL CLI
Version: 3.0.0 — FINAL | Maintainer: RapidXAI | February 2026
Source Skills: Antigravity Awesome Skills (868+ skills) —
github.com/sickn33/antigravity-awesome-skills

> ⚠️ **ABSOLUTE MANDATE:** Every AI agent — regardless of tool, model, or task — MUST
read and execute this SOP in full before writing a single line of code, generating a
single component, or making any architectural decision. This document IS the operating
system of every project. No exceptions. No shortcuts.

🧠 CORE PHILOSOPHY: THE TANDEM SKILL DOCTRINE

> Read this before everything else. This is the single most important concept in this
document.

A single skill is a single expert. Every real project needs a full team.

You are NOT limited to invoking one skill at a time. Invoking a single skill for most
tasks is negligence. You MUST evaluate every task, identify every discipline it
touches, and invoke ALL relevant skills simultaneously or in a defined sequence.

The Three Laws of Skill Invocation

Law 1 — Never use one skill where two or more apply.
If a task touches design AND code, invoke both `@ui-ux-pro-max` AND `@react-patterns`.
If it touches code AND security, invoke both `@typescript-expert` AND
`@api-security-best-practices`. Always.

**Law 2 — Skills within the same layer run in parallel. Skills across layers run in
sequence.**
You can invoke `@frontend-design` + `@react-patterns` + `@tailwind-patterns` at the
same time because they are all frontend skills operating on the same surface. But you
run planning BEFORE building, and testing AFTER building.

Law 3 — Declare your full skill stack before you touch anything.
```

At the start of every session and every major task, output a skill stack declaration (see Phase 0). An undeclared skill is an unused skill.

### ### The Tandem Mental Model

Think of every project as a building with floors. Each floor needs its own specialist team, and no floor can be skipped:

...

PROJECT STACK - ALL FLOORS MUST BE STAFFED

```
|— Floor 6: Business → @brainstorming + @concise-planning + @copywriting
|— Floor 5: Design → @ui-ux-pro-max + @frontend-design + @tailwind-patterns
|— Floor 4: Frontend → @react-best-practices + @react-patterns +
@nextjs-best-practices
|— Floor 3: Backend → @senior-fullstack + @backend-dev-guidelines +
@api-patterns
|— Floor 2: Data → @database-design + @postgres-best-practices + @sql-pro
|— Floor 1: Security → @api-security-best-practices + @backend-security-coder +
@frontend-security-coder
|— Floor 0: Infra → @docker-expert + @deployment-procedures +
@observability-engineer
```

...

You do not need every floor for every task. But you MUST identify which floors are touched and staff them all.

### ### Tandem Invocation - Real Syntax Examples

Invoke multiple skills in a single prompt, explicitly:

...

"Use @ui-ux-pro-max and @react-patterns together to build the hero section.  
Also apply @tailwind-patterns for all styling and @seo-audit for meta structure."

...

...

"Invoke @senior-fullstack alongside @backend-dev-guidelines and @api-patterns  
to design and implement the authentication module, with @api-security-best-practices  
and @auth-implementation-patterns running in parallel for every decision."

...

...

```
"Run @test-driven-development, @e2e-testing-patterns, and @browser-automation
in tandem to produce unit, integration, and E2E tests for the checkout flow."
...
```

```
...
```

```
"Use @brainstorming and @concise-planning together to plan the sprint,
then use @product-manager-toolkit to produce the PRD."
...
```

```

```

## ## 📄 TABLE OF CONTENTS

1. [Phase 0 – Pre-Boot & Skill Stack Declaration] ([#phase-0](#))
2. [Phase 1 – Project Discovery & Intelligence Gathering] ([#phase-1](#))
3. [Phase 2 – Architecture & System Design] ([#phase-2](#))
4. [Phase 3 – Web Design & UI/UX Execution] ([#phase-3](#))
5. [Phase 4 – Senior Code Development] ([#phase-4](#))
6. [Phase 5 – Senior Code Analysis & Review] ([#phase-5](#))
7. [Phase 6 – Security Hardening] ([#phase-6](#))
8. [Phase 7 – Infrastructure & Deployment] ([#phase-7](#))
9. [Phase 8 – Testing & QA] ([#phase-8](#))
10. [Phase 9 – Documentation & Handoff] ([#phase-9](#))
11. [Phase 10 – Post-Deployment & Observability] ([#phase-10](#))
12. [Master Tandem Playbook – Pre-Built Combos by Project Type] ([#tandem-playbook](#))
13. [Full Skill Reference by Bundle] ([#skill-reference](#))
14. [Emergency Protocols] ([#emergency](#))
15. [Project Brain Management] ([#brain](#))
16. [Session Completion Checklist] ([#checklist](#))

```

```

## ## PHASE 0 – PRE-BOOT & SKILL STACK DECLARATION {#phase-0}

```
> **Trigger:** ALWAYS. The very first thing in every session, before any other action.
```

### ### 0.1 – Read the Brain Directory First

```
```bash
```

```
cat .gemini/antigravity/brain/architecture.md 2>/dev/null
```

```
cat .gemini/antigravity/brain/decisions.md 2>/dev/null
```

```
cat .gemini/antigravity/brain/stack.md 2>/dev/null
```

```
cat .gemini/antigravity/brain/skill-tandems.md 2>/dev/null
cat .gemini/antigravity/brain/conventions.md 2>/dev/null
...
```

If brain files exist → load ALL prior decisions and do not contradict them unless explicitly told to.

0.2 – Verify Skill Installation

```
```bash
Check universal path first
test -d ~/.agent/skills && echo "✅ Skills ready at ~/.agent/skills"

If not found, install immediately:
npx antigravity-awesome-skills

Tool-specific fallbacks:
npx antigravity-awesome-skills --claude → .claude/skills/
npx antigravity-awesome-skills --gemini → .gemini/skills/
npx antigravity-awesome-skills --cursor → .cursor/skills/
npx antigravity-awesome-skills --codex → .codex/skills/

If 404 error:
npx github:sickn33/antigravity-awesome-skills
...
```

### ### 0.3 – Mandatory Skill Stack Declaration

Output this block BEFORE any work begins. It is non-negotiable:

...

AGENT ROLE:	[Full-Stack Dev / Architect / Designer / All]
PROJECT:	[Project Name]
TECH STACK:	[Next.js / Python / N8N / Supabase / etc.]
SESSION GOAL:	[Precise description of what we build today]
ACTIVE SKILL STACK (TANDEM GROUPS):	
└─ Planning:	@brainstorming + @concise-planning
└─ Design:	@ui-ux-pro-max + @frontend-design
└─ Frontend:	@react-patterns + @tailwind-patterns
└─ Backend:	@senior-fullstack + @api-patterns

```

|| └─ Security: @api-security-best-practices + @backend-security-coder ||
|| └─ Testing: @test-driven-development + @e2e-testing-patterns ||
|| └─ Deploy: @deployment-procedures + @docker-expert ||
|| ||
|| RULE: Minimum 3 skills active. Minimum 2 invoked in tandem ||
|| for any non-trivial task. ||
||=====||

```

...

### ### 0.4 – Establish Operating Context

Before writing anything, answer these internally:

...

- What is the business goal? (Not just the technical ask)
- Which floors of the project stack does this session touch?
- What constraints exist? (Budget, timeline, API limits, hosting)
- Who is the end user? (B2B / B2C / Developer / Non-technical)
- What integrations are required?

...

---

## ## PHASE 1 – PROJECT DISCOVERY & INTELLIGENCE GATHERING {#phase-1}

> **\*\*Trigger:\*\*** New project, new major feature, or first session on an unfamiliar codebase.

### ### 1.1 – Codebase Scan

```

```bash
find . -type f \( -name "*.ts" -o -name "*.tsx" -o -name "*.js" -o -name "*.py" \) |
head -80
cat package.json 2>/dev/null
cat requirements.txt 2>/dev/null
cat .env.example 2>/dev/null
ls -la
...

```

1.2 – Planning Tandem (Always These Three Together)

...

```
Use @brainstorming and @concise-planning together to analyze the project goal
and produce a prioritized, structured development plan.
Simultaneously use @product-manager-toolkit to define acceptance criteria
and RICE-score the feature priorities.
...

These three always run together. `@brainstorming` generates ideas →
`@concise-planning` structures them → `@product-manager-toolkit` turns them into a
shippable spec.
```

1.3 – Business Context Alignment

Resolve all of these before writing a single line of code:

- What is the core user journey from landing to conversion?
- What does "done" look like for this milestone?
- What are the hard constraints? (deadline, spend cap, team size)
- What existing systems must this connect to?

1.4 – Preliminary Risk Scan (Dual-Skill)

```
...

Use @vulnerability-scanner and @api-security-best-practices together
to perform a preliminary dual-layer risk scan on the project type
and any third-party integrations involved.
...

---
```

PHASE 2 – ARCHITECTURE & SYSTEM DESIGN {#phase-2}

> ****Trigger:**** New project, major refactor, or any feature touching more than 3 files.

2.1 – Architecture Triple Invocation

```
...

Use @senior-architect, @architecture-patterns, and @architecture-decision-records
together:
- @senior-architect: owns high-level system design, scaling strategy, component
boundaries
- @architecture-patterns: selects the right pattern (Clean Architecture, DDD,
Hexagonal, Microservices)
```

```
- @architecture-decision-records: produces a formal, dated ADR for every major decision

All three run in tandem - the architect's decisions feed directly into the pattern selection,

which feeds directly into the ADR document.

...
```

2.2 - Stack Selection Criteria (Non-Negotiable)

```
| Criterion | Minimum Standard |
|---|---|
| Scalability | Handles 10x current load without architecture change |
| Maintainability | Any developer understands the codebase in <30 min |
| Cost | Monthly infra cost documented before first commit |
| Security | Auth, RBAC, and data isolation designed at layer 0 |
| Developer Experience | Hot reload, strict typing, linting active from day 1 |
```

2.3 - API Contract Design (Tandem)

```
...

Use @senior-fullstack, @api-patterns, and @typescript-expert together.
@api-patterns selects the API style (REST vs GraphQL vs tRPC).
@senior-fullstack designs the contract shape and data flow.
@typescript-expert defines strict TypeScript types and interfaces.
If Python: swap @typescript-expert for @python-patterns or @fastapi-pro.

...
```

2.4 - Database Schema Design

```
...

Use @database-design and @postgres-best-practices together for all schema work.
@database-design: normalization, relationships, ORM selection, indexing strategy.
@postgres-best-practices: Postgres-specific optimization, query patterns, RLS policies.

For Supabase projects: also invoke @supabase (if available) for native Supabase patterns.

...
```

2.5 - Write Architecture to Brain

After this phase, IMMEDIATELY write all decisions:

```

```bash
mkdir -p .gemini/antigravity/brain/
Write: architecture summary, stack decisions, ADRs, API contracts to brain files
```

---

## PHASE 3 – WEB DESIGN & UI/UX EXECUTION {#phase-3}

> **Trigger:** Any frontend work – landing pages, dashboards, SaaS UIs, client websites.

### 3.1 – The Designer + Developer Core Duo (MANDATORY PAIRING)

> This is the most fundamental tandem in all frontend work. These two skills are NEVER separated.

...

Use @ui-ux-pro-max and @react-patterns together for ALL frontend work.
This pairing is non-negotiable. One without the other produces incomplete output.

@ui-ux-pro-max owns:
- Design system (tokens: color, spacing, typography, radius, shadow)
- Visual hierarchy and layout rhythm
- Component visual states (default, hover, active, disabled, error)
- Mobile-first responsive breakpoints
- Accessibility (contrast ratios, ARIA, focus management)

@react-patterns owns:
- Component architecture and file structure
- Hook design and state management
- Performance (memoization, lazy loading, code splitting)
- Composability and reusability patterns
...

### 3.2 – Extended Frontend Stack for Full Pages

For any complete page (not just a component), extend the core duo into a full stack:

...

Use @ui-ux-pro-max + @frontend-design + @react-patterns + @react-best-practices
+ @tailwind-patterns + @nextjs-best-practices together.

```

Role breakdown:

- @ui-ux-pro-max: Design system, tokens, visual language
 - @frontend-design: Aesthetics, layout composition, colour usage
 - @react-patterns: Component structure and React best practices
 - @react-best-practices: Performance optimization and React 19+ patterns
 - @tailwind-patterns: Utility-first CSS with Tailwind v4
 - @nextjs-best-practices: App Router, RSC, Server Actions, route handlers
- ...

3.3 – Conversion & Marketing Pages (Full 6-Skill Stack)

Landing pages, pricing pages, and any conversion-critical surface require all six:

...

Use @ui-ux-pro-max + @frontend-design + @react-patterns + @tailwind-patterns + @copywriting + @seo-audit + @form-cro together.

- @copywriting: All user-facing text – headlines, CTAs, microcopy, value props
 - @seo-audit: Meta structure, semantic HTML, structured data, page titles
 - @form-cro: Form design optimized for conversion and reduced friction
- ...

3.4 – Immersive / Animated Design

For scroll experiences, 3D elements, or interactive portfolios:

...

Use @ui-ux-pro-max + @scroll-experience + @3d-web-experience + @react-patterns together.

- @scroll-experience: Scroll-driven animations and parallax patterns
 - @3d-web-experience: Three.js and React Three Fiber integration
- ...

3.5 – Mobile Design

...

Use @ui-ux-pro-max + @mobile-design + @react-patterns together for all mobile-first work.

@mobile-design: Touch targets, gesture handling, viewport scaling, iOS/Android quirks.

...

3.6 – Design Tokens (Mandatory Before Any Styled Component)

Before writing a single styled component, establish:

```
``typescript
// tokens.ts – define FIRST, use everywhere
const tokens = {
  colors: { primary, secondary, accent, surface, error, success },
  spacing: { xs: 4, sm: 8, md: 16, lg: 24, xl: 32, xxl: 48 },
  typography: { fontFamily, fontSizes, fontWeights, lineHeights },
  radii: { sm: 4, md: 8, lg: 16, full: 9999 },
  shadows: { sm, md, lg, xl }
}
...

```

3.7 – Accessibility Standards (Non-Negotiable)

Every interactive element MUST have:

- `aria-label` or visible label text
- Keyboard navigability (`tabIndex`, `focus-visible` styles)
- Color contrast ratio $\geq 4.5:1$ (WCAG AA minimum)
- Screen reader compatible semantic markup

3.8 – Responsive Breakpoints (Test All Before Marking Done)

- `sm` 640px – mobile
- `md` 768px – tablet
- `lg` 1024px – laptop
- `xl` 1280px – desktop
- `2xl` 1536px – wide monitor

PHASE 4 – SENIOR CODE DEVELOPMENT {#phase-4}

> ****Trigger:**** Anytime code is being written – features, fixes, refactors, automation, scripts.

4.1 – Pre-Code Mandatory Checklist

Before writing ANY code, confirm all are true:

- [] Architecture decisions are clear and in brain files
- [] Types/interfaces are defined before implementation (TypeScript) or type hints written (Python)
- [] The function/component has a single, clear responsibility
- [] Edge cases are identified and will be handled explicitly
- [] The active skill stack covers every discipline this code touches
- [] The code can be tested in isolation

4.2 – Full-Stack Feature Development – The 7-Layer Stack

When building a complete feature end-to-end, all seven layers must be staffed:

...

Layer 1 – Planning:

Use @brainstorming + @concise-planning to spec the feature before touching code.

Layer 2 – Design:

Use @ui-ux-pro-max + @frontend-design + @react-patterns for all UI surfaces.

Layer 3 – Frontend:

Use @react-best-practices + @tailwind-patterns + @nextjs-best-practices.

Layer 4 – Backend:

Use @senior-fullstack + @backend-dev-guidelines + @api-patterns.

Layer 5 – Data:

Use @database-design + @postgres-best-practices.

Layer 6 – Security:

Use @api-security-best-practices + @backend-security-coder +

@frontend-security-coder.

(Security is ALWAYS active – it does not get deactivated for "quick" features.)

Layer 7 – Testing:

Use @test-driven-development + @e2e-testing-patterns.

...

4.3 – Language & Framework Tandems

****TypeScript / React / Next.js:****

...

Use @typescript-expert + @react-best-practices + @react-patterns

```

    + @nextjs-best-practices + @nextjs-app-router-patterns together.
    ...

**Python / FastAPI:**
    ...

Use @python-pro + @python-patterns + @fastapi-pro
    + @async-python-patterns + @python-testing-patterns together.
    ...

**Node.js API:**
    ...

Use @senior-fullstack + @backend-dev-guidelines + @typescript-expert
    + @nodejs-best-practices + @api-security-best-practices together.
    ...

**AI Agent / LLM System:**
    ...

Use @ai-agents-architect + @prompt-engineering + @rag-engineer
    + @langgraph + @llm-app-patterns + @context-window-management together.
All six are required for any non-trivial AI system.
    ...

**N8N / Make / Automation Workflow:**
    ...

Use @workflow-automation (if available) + @api-patterns
    + @api-security-best-practices + @systematic-debugging together.
    ...

**Game Development (Unity/Unreal):**
    ...

Use @unity-developer + @game-design + @3d-games together for 3D.
Use @unity-developer + @game-design + @2d-games together for 2D.
Also invoke @algorithmic-art for procedural asset generation.
    ...

```

4.4 – Code Quality Standards (Hard Rules)

| Standard | Requirement |
|-----------------------|---|
| Function length | Maximum 40 lines per function |
| Cyclomatic complexity | Maximum 10 |
| Naming | Self-documenting – no single-letter variables except loop iterators |

```
Comments	Only explain "why", never "what" – the code explains what
Error handling	Every async function has try/catch or an error boundary
Logging	Structured logs with context – no raw `console.log` in production
Secrets	ALL keys in `.env` – never hardcoded, never committed, never logged
```

4.5 – API Integration Standards

```
...
```

```
Use @api-security-best-practices + @auth-implementation-patterns together
for every external API integration.
```

```
...
```

- API keys: `.env` always – if it's hardcoded, it's wrong
- Webhook payloads: validate signature before processing
- External calls: retry with exponential backoff
- Rate limits: respect provider limits, implement client-side throttling

4.6 – Stripe / Payments

```
...
```

```
Use @stripe-integration + @api-security-best-practices + @pci-compliance together.
No payment integration is complete without all three.
```

```
...
```

4.7 – Communications (Twilio, SMS, Voice)

```
...
```

```
Use @twilio-communications + @api-security-best-practices together.
```

```
...
```

```
---
```

PHASE 5 – SENIOR CODE ANALYSIS & REVIEW {#phase-5}

```
> **Trigger:** Code review, PR review, debugging session, refactoring, or taking over
an existing codebase.
```

5.1 – Full Code Audit – Mandatory 5-Skill Tandem

```
...
```

```
Use @senior-architect + @lint-and-validate + @vulnerability-scanner
+ @code-review-checklist + @systematic-debugging together.
```

Each skill produces its own report. Reports are then merged:

- @senior-architect: Architectural quality, technical debt, scaling risks
 - @lint-and-validate: Static analysis, code style, type errors
 - @vulnerability-scanner: OWASP Top 10, dependency CVEs, injection risks
 - @code-review-checklist: PR-level checklist, logic errors, missed edge cases
 - @systematic-debugging: Root cause analysis for any identified failures
- ...

5.2 – Performance Analysis

...

Use @performance-engineer + @react-best-practices together to identify:

...

- N+1 database queries
- Blocking main thread operations
- Unnecessary re-renders (React DevTools Profiler)
- Unoptimized bundle size (``next build && next analyze``)
- Missing image optimization

****Lighthouse targets:**** ≥ 90 for Performance, Accessibility, Best Practices, SEO

5.3 – Technical Debt Classification

Label all findings before handing off:

| Label | Definition | Required Action |
|-------------|---|-------------------------|
| --- --- --- | | |
| 🔴 CRITICAL | Security vulnerability or data loss risk | Block merge – fix first |
| 🟡 HIGH | Broken user flow or major performance issue | Fix in current sprint |
| 🟡 MEDIUM | Code smell, missing test, or anti-pattern | Fix in next sprint |
| 🟢 LOW | Style inconsistency or minor refactor opportunity | Backlog |

5.4 – Dependency Audit

```bash

npm audit --audit-level=high

# Python:

pip-audit

...

```

`high` or `critical` severity = **merge blocked until resolved**.

PHASE 6 – SECURITY HARDENING {#phase-6}

> **Trigger:** Before ANY production deployment. After any auth change. Weekly on all
live products.

6.1 – The Security Trio (Always All Three, Never Fewer)

...

Use @api-security-best-practices + @backend-security-coder + @frontend-security-coder
together.

These three are NEVER invoked individually for production security work.
They always run as a trio – API layer, server layer, and client layer simultaneously.

For deep audits, extend to the full Security Engineer stack:
Use @ethical-hacking-methodology + @top-web-vulnerabilities + @security-auditor
 + @vulnerability-scanner + @api-security-best-practices together.
...

6.2 – Authentication & Authorization Rules

...

Use @auth-implementation-patterns for all auth implementation.
...

- Auth is NEVER built from scratch – use Supabase Auth, NextAuth, Clerk, or Auth0
- Every route has explicit RBAC defined before any code is written
- JWT tokens: 15-minute access, 7-day refresh, never localStorage
- All sensitive operations require explicit re-authentication

6.3 – Data Protection

- PII is never logged, anywhere, ever
- PII fields are encrypted at rest
- All transmission uses HTTPS (TLS 1.3 minimum)
- File uploads: validate type AND magic bytes (not just file extension)
- ALL SQL queries: parameterized statements only – no string interpolation

6.4 – Mandatory Security Headers

```

```
```http
Content-Security-Policy: default-src 'self'; script-src 'self'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
Referrer-Policy: no-referrer-when-downgrade
Permissions-Policy: camera=(), microphone=(), geolocation=()
```
```

### ### 6.5 – Payments Security

```
```
Use @pci-compliance + @stripe-integration + @api-security-best-practices together.
Never handle raw card data – always use Stripe Elements or Stripe Checkout.
```
```

---

## ## PHASE 7 – INFRASTRUCTURE & DEPLOYMENT {#phase-7}

> **\*\*Trigger:\*\*** Before any deployment – staging or production.

### ### 7.1 – Deployment Tandem

```
```
Use @deployment-procedures + @docker-expert + @observability-engineer together
for all deployments.
- @deployment-procedures: safe rollout strategy, canary/blue-green, rollback plan
- @docker-expert:          container build optimization, multi-stage Dockerfiles,
compose
- @observability-engineer: monitoring setup, alerting thresholds, health checks
For AWS: add @aws-serverless as a fourth skill.
For K8s: add @kubernetes-architect as a fourth skill.
For IaC: add @terraform-specialist as a fifth skill.
```
```

### ### 7.2 – Environment Management

Every project runs three environments with no exceptions:

| Environment | Purpose | Auto-Deploy | Approval Required |
|-------------|---------|-------------|-------------------|
|-------------|---------|-------------|-------------------|

```
|---|---|---|---|
```

```
`development`	Local dev, experiments	No	No
`staging`	Pre-prod testing, client demos	Yes (on PR)	No
`production`	Live user traffic	No	Yes – manual only
```

### ### 7.3 – Pre-Deployment Checklist

- [ ] All environment variables set in deployment platform (NOT in `.env` file committed to git)
- [ ] Database migrations are backwards-compatible (can roll back without data loss)
- [ ] Health check endpoint active and returns 200 (`/api/health`)
- [ ] Error tracking configured (Sentry or equivalent)
- [ ] Rollback procedure documented AND tested
- [ ] Rate limiting active on all public-facing endpoints
- [ ] Security headers verified

### ### 7.4 – CI/CD Pipeline – Required Order

1. `lint` – fail fast on style and formatting
2. `type-check` – TypeScript `tsc --noEmit` or mypy
3. `test` – unit + integration
4. `build` – production build must succeed
5. `security-scan` – `npm audit --audit-level=high`
6. `deploy` – only on `main` or tagged release branch, after human approval for production

```

```

## ## PHASE 8 – TESTING & QA {#phase-8}

> **\*\*Trigger:\*\*** After every feature. Before every PR. Before every deployment.

### ### 8.1 – The QA Skill Stack (All Five Active Together)

```
...
```

Use @test-driven-development + @systematic-debugging + @browser-automation  
+ @e2e-testing-patterns + @test-fixing together.

Role breakdown:

- @test-driven-development: Red/green/refactor cycle – write tests first
- @systematic-debugging: Root cause analysis when tests fail
- @browser-automation: Playwright-based E2E and browser automation

```

- @e2e-testing-patterns: Reliable, non-flaky E2E test suite design
- @test-fixing: Systematic approach to fixing broken tests after
refactors
...

8.2 – Test Coverage Requirements

Layer	Minimum Coverage	Test Type
Pure utility functions	100%	Unit
API routes / handlers	90%	Integration
Critical UI components	80%	Component
Critical user journeys	100%	E2E

8.3 – Non-Negotiable E2E Tests (Must Exist Before Production)

- User registration and email verification
- Login, logout, session expiry
- Core conversion action (signup CTA, purchase, booking, form submission)
- Payment flow (if applicable – full Stripe test mode run)
- Data submission and retrieval round-trip
- Error states (network failure, validation errors, 404, 500)
- Empty states (new user, no data, no results)

8.4 – Performance Benchmarks

```bash
npx lhci autorun
# Load test:
npx autocannon -c 100 -d 30 https://staging.yourapp.com/api/health
...

---

## PHASE 9 – DOCUMENTATION & HANDOFF {#phase-9}

> **Trigger:** After every major feature. At project completion. Before any handoff.

### 9.1 – Documentation Tandem

...

Use @documentation-templates + @changelog-automation together.

```

```

@documentation-templates: standardized docs structure and handoff format.
@changelog-automation: keep release notes consistent and up to date automatically.
...

### 9.2 – Mandatory Documentation Set

| Document | Location | Owner Skill | Contents |
|---|---|---|---|
| `README.md` | Root | `@documentation-templates` | Setup, run, deploy, env vars |
| `ARCHITECTURE.md` | `/docs` | `@architecture-decision-records` | System design,
ADRs, data flow |
| `API.md` | `/docs` | `@api-patterns` | All endpoints, payloads, auth, error codes |
| `ENVIRONMENT.md` | `/docs` | `@deployment-procedures` | All env vars with
descriptions and examples |
| `CHANGELOG.md` | Root | `@changelog-automation` | Version history and migration
notes |
| `brain/` | `.gemini/antigravity/brain/` | Agent memory | Persistent decisions, skill
tandems, session logs |

### 9.3 – Code-Level Documentation Standards

- Every exported function: JSDoc comment or Python docstring
- Every API endpoint: OpenAPI/Swagger annotation
- Every complex algorithm: plain-English explanation comment above it
- Every `TODO`: includes `@owner-name` and a GitHub issue number

---

## PHASE 10 – POST-DEPLOYMENT & OBSERVABILITY {#phase-10}

> Trigger: Immediately after production deployment. Ongoing weekly cadence.

### 10.1 – Observability Stack (Minimum Viable Production)

...

Use @observability-engineer + @distributed-tracing + @slo-implementation together
for production monitoring setup.

- @observability-engineer: full monitoring stack, alerting, dashboards
- @distributed-tracing: request tracing across microservices/edge functions
- @slo-implementation: Service Level Objectives and error budget tracking
...

```

10.2 – Required Monitoring Stack

- ****Error Tracking:**** Sentry (free tier covers most projects)
- ****Uptime Monitoring:**** UptimeRobot or Better Uptime
- ****Performance:**** Vercel Analytics or Plausible
- ****Log Aggregation:**** Axiom, Logtail, or Vercel Logs

10.3 – Weekly Maintenance Skills

...

Mondays: Use @vulnerability-scanner to run weekly dependency and security scan.

Monthly: Use @seo-audit to run SEO health check on all client-facing properties.

Monthly: Use @performance-engineer to run performance regression check.

...

10.4 – Alerting Thresholds

Metric	Warning Threshold	Critical Threshold
Error rate	>1%	>5%
P99 API latency	>2s	>5s
Uptime	<99.9%	<99%
Failed deployments	Any	Any
Security scan findings	Any HIGH	Any CRITICAL

⚡ MASTER TANDEM PLAYBOOK – PRE-BUILT COMBOS BY PROJECT TYPE {#tandem-playbook}

> Copy any of these directly into your agent prompt. These are the correct skill combinations for each common project type.

🌐 Landing Page / Marketing Site

...

Use @ui-ux-pro-max + @frontend-design + @react-patterns + @tailwind-patterns
+ @copywriting + @seo-audit + @form-cro together.

...

📊 SaaS Dashboard / Admin Panel

...

Use @ui-ux-pro-max + @react-best-practices + @react-patterns + @nextjs-best-practices
+ @senior-fullstack + @database-design + @postgres-best-practices

```
+ @api-security-best-practices together.
...

### 🗝️ Authentication System
...

Use @auth-implementation-patterns + @backend-security-coder + @frontend-security-coder
  + @typescript-expert + @test-driven-development together.
...

### 🗣️ AI Chatbot / Agent System
...

Use @ai-agents-architect + @prompt-engineering + @rag-engineer + @langgraph
  + @llm-app-patterns + @context-window-management + @langfuse together.
...

### 🇪🇺 E-Commerce / Payment Flow
...

Use @stripe-integration + @pci-compliance + @api-security-best-practices
  + @ui-ux-pro-max + @copywriting + @form-cro together.
...

### ⚙️ N8N / Automation Workflow Build
...

Use @api-patterns + @api-security-best-practices + @systematic-debugging
  + @backend-dev-guidelines together.
...

### 🎮 Game Development (Unity / Unreal)
...

For 3D: Use @unity-developer + @game-design + @3d-games + @algorithmic-art together.
For 2D: Use @unity-developer + @game-design + @2d-games + @algorithmic-art together.
...

### 📱 Mobile App (React Native / Flutter)
...

Use @mobile-developer + @react-native-architecture + @mobile-design
  + @api-security-best-practices + @app-store-optimization together.
...

### 🔍 Full Codebase Audit
...

Use @senior-architect + @vulnerability-scanner + @lint-and-validate
```

```
+ @code-review-checklist + @security-auditor + @performance-engineer together.
...

### 🚀 Full SaaS MVP (Ship in 2 Weeks)
...

Week 1 - Build:
  Use @brainstorming + @concise-planning + @product-manager-toolkit (Day 1: spec)
  Use @ui-ux-pro-max + @react-patterns + @tailwind-patterns + @nextjs-best-practices
  (Days 2-4: frontend)
  Use @senior-fullstack + @database-design + @api-security-best-practices (Days 2-4:
  backend)

Week 2 - Ship:
  Use @test-driven-development + @e2e-testing-patterns + @browser-automation (Days 8-9:
  tests)
  Use @api-security-best-practices + @backend-security-coder (Day 10: security
  hardening)
  Use @deployment-procedures + @docker-expert + @observability-engineer (Days 11-14:
  deploy)
...

### 🛡️ Security Audit & Hardening
...

Use @ethical-hacking-methodology + @top-web-vulnerabilities + @vulnerability-scanner
  + @api-security-best-practices + @backend-security-coder + @frontend-security-coder
  + @security-auditor together.
...

### 📈 Growth & Conversion Optimization
...

Use @seo-audit + @programmatic-seo + @analytics-tracking + @ab-test-setup
  + @copywriting + @form-cro + @content-creator together.
...

### 🏢 Enterprise / DDD Architecture
...

Use @domain-driven-design + @ddd-strategic-design + @ddd-tactical-patterns
  + @cqrs-implementation + @event-sourcing-architect + @microservices-patterns
  together.
...

---
```



```

@frontend-design          - UI guidelines and aesthetics
@react-best-practices      - React & Next.js performance optimization
@react-patterns           - Modern React patterns and principles
@nextjs-best-practices    - Next.js App Router patterns
@tailwind-patterns       - Tailwind CSS v4 styling
@form-cro                 - Forms optimized for conversion
@seo-audit                - Technical SEO health checks

```

⚡ Full-Stack Developer Pack

```

@senior-fullstack        - Complete full-stack development guide
@frontend-developer      - React 19+ and Next.js 15+ expertise
@backend-dev-guidelines  - Node.js/Express/TypeScript patterns
@api-patterns            - REST vs GraphQL vs tRPC selection
@database-design         - Schema design and ORM selection
@stripe-integration      - Payments and subscriptions

```

🤖 Agent Architect Pack

```

@agent-evaluation        - Test and benchmark your agents
@langgraph               - Stateful agent workflows
@mcp-builder             - Create custom MCP tools
@prompt-engineering      - LLM prompt design and optimization
@ai-agents-architect     - Design autonomous AI agents
@rag-engineer            - RAG systems with vector search

```

🧠 LLM Application Developer Pack

```

@llm-app-patterns       - Production-ready LLM patterns
@rag-implementation     - Retrieval-Augmented Generation
@prompt-caching          - Cache strategies for LLM prompts
@context-window-management - Manage LLM context efficiently
@langfuse                - LLM observability and tracing

```

🐍 Python Pro Pack

```

@python-pro             - Python 3.12+ modern features

```

```
@python-patterns          - Idiomatic Python code
@fastapi-pro               - High-performance async APIs
@fastapi-templates         - Production-ready FastAPI
@async-python-patterns     - Python asyncio mastery
@python-testing-patterns   - pytest comprehensive testing
...
```

TypeScript & JavaScript Pack

```
...
@typescript-expert         - TypeScript mastery, strict mode, generics
@javascript-pro            - Modern JavaScript with ES6+
@react-best-practices      - React performance optimization
@nodejs-best-practices     - Node.js development principles
@nextjs-app-router-patterns - Next.js 14+ App Router
...
```

Security Engineer Pack

```
...
@ethical-hacking-methodology - The methodology for ethical hacking
@top-web-vulnerabilities     - OWASP-aligned vulnerability taxonomy
@security-auditor            - Comprehensive security audits
@vulnerability-scanner       - Advanced vulnerability analysis
@cloud-penetration-testing   - AWS/Azure/GCP security
...
```

Security Developer Pack

```
...
@api-security-best-practices - Secure API design patterns
@auth-implementation-patterns - JWT, OAuth2, session management
@backend-security-coder      - Secure backend coding
@frontend-security-coder     - XSS prevention, client-side security
@cc-skill-security-review    - Security checklist for features
@pci-compliance              - Payment card security standards
...
```

QA & Testing Pack

```
...
@test-driven-development     - Red, Green, Refactor cycle
@systematic-debugging        - Debug like Sherlock Holmes
@browser-automation          - E2E testing with Playwright
@e2e-testing-patterns        - Reliable E2E test suites
@code-review-checklist        - Catch bugs in PRs
```

```
@test-fixing          - Fix failing tests systematically
...

```

☁️ DevOps & Cloud Pack

```
...
@docker-expert        - Containers and multi-stage builds
@aws-serverless       - Lambda, DynamoDB, serverless
@kubernetes-architect - K8s architecture and GitOps
@terraform-specialist - Infrastructure as Code
@deployment-procedures - Safe rollout strategies
@bash-linux           - Terminal wizardry
...

```

📊 Observability & Monitoring Pack

```
...
@observability-engineer - Comprehensive monitoring systems
@distributed-tracing    - Track requests across microservices
@slo-implementation     - Service Level Objectives
@incident-responder     - Rapid incident response
@postmortem-writing     - Blameless postmortems
@performance-engineer   - Application performance optimization
...

```

🦄 Startup Founder Pack

```
...
@product-manager-toolkit - RICE prioritization, PRD templates
@competitive-landscape   - Competitor analysis
@launch-strategy         - Product launch planning
@copywriting             - Marketing copy that converts
@stripe-integration      - Get paid from day one
...

```

📈 Marketing & Growth Pack

```
...
@content-creator        - SEO-optimized marketing content
@seo-audit              - Technical SEO health checks
@programmatic-seo       - Pages at scale
@analytics-tracking     - GA4/PostHog setup
@ab-test-setup          - Validated learning experiments
@email-sequence         - Automated email campaigns
...

```

🎮 Game Dev Pack

...

@game-design	- Mechanics and game loops
@2d-games	- Sprites and 2D physics
@3d-games	- 3D models and shaders
@unity-developer	- Unity 6 LTS development
@godot-gdscript-patterns	- Godot 4 GDScript patterns
@algorithmic-art	- Generate assets with code

...

🏗️ Architecture & Design Pack

...

@senior-architect	- Comprehensive software architecture
@architecture-patterns	- Clean Architecture, DDD, Hexagonal
@microservices-patterns	- Microservices architecture
@event-sourcing-architect	- Event sourcing and CQRS
@architecture-decision-records	- Document technical decisions

...

🧱 DDD & Evented Architecture Pack

...

@domain-driven-design	- Strategic to tactical DDD routing
@ddd-strategic-design	- Bounded contexts, ubiquitous language
@ddd-tactical-patterns	- Aggregates, value objects, repositories
@cqrs-implementation	- Read/write model separation
@event-store-design	- Event persistence and replay
@saga-orchestration	- Cross-context transaction coordination
@projection-patterns	- Materialized read models from events

...

🛠️ OSS Maintainer Pack

...

@commit	- High-quality conventional commits
@create-pr	- PR creation with review-ready context
@changelog-automation	- Consistent release notes
@git-advanced-workflows	- Rebase, cherry-pick, bisect, recovery
@documentation-templates	- Standardize docs and handoffs

...

🔗 Integrations Pack

...

@stripe-integration	- Payments and subscriptions
---------------------	------------------------------

```
@twilio-communications      - SMS, voice, WhatsApp
@hubspot-integration        - CRM integration
@algolia-search             - Search implementation
@analytics-tracking         - GA4/PostHog setup
...

---

## 🚨 EMERGENCY PROTOCOLS {#emergency}

### Production Is Down

...

1. DO NOT make changes without a documented rollback plan.
2. Use @observability-engineer + @systematic-debugging + @incident-responder together
   to diagnose the failure and coordinate response.
3. Check logs FIRST — never guess the cause.
4. Roll back to last known good deployment.
5. Use @postmortem-writing to write a blameless post-mortem within 24 hours.
...

### Security Breach Suspected

...

1. IMMEDIATELY revoke all active API keys, sessions, and tokens.
2. Use @vulnerability-scanner + @api-security-best-practices
   + @ethical-hacking-methodology together to assess the full attack surface.
3. Do not discuss the breach publicly until investigation is complete.
4. Notify affected users within 72 hours (GDPR/compliance requirement).
5. Use @security-auditor to document findings and remediation.
...

### Runaway API Costs (Google AI Studio / Gemini)

...

1. Immediately add hard spend cap in Google Cloud Console.
2. Use @api-security-best-practices + @observability-engineer together
   to identify the source of runaway calls and add rate limiting.
3. Enable billing alerts at 50%, 80%, and 100% of budget threshold.
4. Review all public endpoints for missing authentication.
...
```

Critical Bug in Production

```
...  
Use @systematic-debugging + @incident-responder together for active debugging.  
Use @test-driven-development to write a regression test before deploying the fix.  
Never deploy a fix without a test that catches it.  
...
```

🧠 PROJECT BRAIN MANAGEMENT {#brain}

Brain Directory Structure

```
...  
.gemini/antigravity/brain/  
├─ architecture.md      # System design decisions and ADRs  
├─ stack.md             # Tech stack with justifications and alternatives rejected  
├─ decisions.md         # Key decisions with rationale, date, and owner  
├─ conventions.md       # Naming, folder structure, code conventions  
├─ integrations.md      # All external APIs, key structure, webhook endpoints  
├─ known-issues.md      # Known bugs, technical debt, workarounds  
├─ skill-tandems.md     # ★ Successful skill combos discovered in this project  
└─ session-log.md       # Summary of what was accomplished each session  
...
```

> `skill-tandems.md` is critical. Every time a skill combination works exceptionally well for this specific project, log it here so every future agent session can build on it without rediscovering it.

Brain Read Protocol (Every Session Start)

```
```bash  
Run this before anything else in every session
for f in architecture stack decisions conventions integrations skill-tandems; do
 echo "=== $f ===" && cat ".gemini/antigravity/brain/$f.md" 2>/dev/null || echo "(not
yet written)"
done
...
```

### ### Brain Write Protocol (Every Session End and After Major Decisions)

Write to brain after:

- Every architectural decision (with rationale and date)
- Every new convention established
- Every new external integration added
- Every successful new skill tandem discovered
- Every significant bug found AND its root cause
- Every session end (what was accomplished, what is next)

---

##  SESSION COMPLETION CHECKLIST {#checklist}

Before ending ANY work session, every item must be checked:

#### **\*\*Code Quality\*\***

- [ ] Lint and type checks pass (`npm run lint && tsc --noEmit`)
- [ ] No secrets or API keys anywhere in committed code
- [ ] All new code has corresponding tests
- [ ] No `console.log` statements in production paths

#### **\*\*Documentation\*\***

- [ ] Brain files updated with session summary and any new decisions
- [ ] `brain/skill-tandems.md` updated with effective skill combinations used today
- [ ] CHANGELOG.md updated with what changed
- [ ] Outstanding TODOs have GitHub issue numbers assigned

#### **\*\*Deployment (if applicable)\*\***

- [ ] Staging deployment verified and tested
- [ ] Security headers confirmed active
- [ ] Monitoring alerts confirmed active
- [ ] Rollback procedure confirmed available

#### **\*\*Handoff\*\***

- [ ] Next session's starting point is documented in `brain/session-log.md`
- [ ] Any blockers or open questions are written down explicitly

---

##  RAPIDXAI PROJECT TYPE QUICK-SELECT

For RapidXAI client work specifically, use these pre-selected stacks:

Client Type	Lead Skills	Tandem	Support Skills
HVAC / Plumbing / Solar	`@ai-agents-architect + @prompt-engineering`		`@ui-ux-pro-max + @senior-fullstack + @stripe-integration`
Real Estate Agency	`@rag-engineer + @langgraph`		`@react-best-practices + @seo-audit + @copywriting`
E-Commerce / Shopify	`@stripe-integration + @frontend-developer`		`@ui-ux-pro-max + @form-cro + @seo-audit`
Dental / Hair Salon	`@ai-agents-architect + @twilio-communications`		`@ui-ux-pro-max + @copywriting + @seo-audit`
Tour Operators	`@senior-fullstack + @database-design`		`@ui-ux-pro-max + @react-patterns + @stripe-integration`
SaaS Product (RapidXAI)	`@senior-fullstack + @ai-agents-architect`		`@react-best-practices + @api-security-best-practices + @product-manager-toolkit`

---

*\*This SOP is a living document. Every agent that reads it is expected to follow it with zero deviation unless explicitly overridden by the project owner. It is updated as the project evolves, new skills are discovered, and better tandem combinations are found.\**

*\*Built for RapidXAI. Powered by Antigravity Awesome Skills (868+ skills – [github.com/sickn33/antigravity-awesome-skills](https://github.com/sickn33/antigravity-awesome-skills)). February 2026.\**

Emergency Fixes:

Emergency Fixes:

In case you cannot see the Shopify Store Products tell it this:

The shop is non-functional. Please fix it. Thank you

Can not see the products and cannot see the card. I cannot just show the products even though I have four products in my shopper fest too. I cannot see them in the shop, in the cart section so please fix it. Also all the other pages are not redirecting anywhere and they're giving errors like 404 file not found so please fucking fix that as well. Thank you

IN case All the pages are not populated.

Populate all the pages with clean aesthetics and use nanobarana pro to generate images if required. Make sure you populate all the pages and make sure they look fucking insane and fucking good and yeah they just match the same vibe and aesthetic of the main website. Make sure it looks premium, high-end, solo components for any company to purchase. Thank you

# Prompts I Spoke

## Prompts I Spoke

Prompts that I spoke out during the video:

you can get it all here

For Website Creation:

Here is a website which I really like. I want to recreate this website, and I want you to give me a prompt for Google anti-gravity so that I can make a complete Google anti-gravity website similar to this website from scratch. If I give this image as a reference give me a complete Google anti-gravity prompt to create this website with all the different coding elements and code snippets required to replicate this from scratch completely. The difference should be that this website would feature multiple pages, but the aesthetics, the colour, the style, the feel, the vibe should be of this website. Thank you.

The output prompt should be highly detailed and untruncated with as much detail as possible to make it as easy as possible for google antigravity to replicate the website

Image Prompt:

```
{
 "prompt": "Aesthetic mirror selfie of a young man in a dimly lit modern bedroom, wearing a black oversized sweatshirt, holding a smartphone. A white cat is sitting on his shoulder and licking his hair affectionately. Soft warm overhead lighting from a hanging lamp creates a cozy mood. Slightly messy mirror with smudges and dust visible for a candid, realistic feel. Neutral expression, soft facial features, natural hair. Minimalist interior background with a door and light-colored walls. Subtle grain and low-light noise for a film-like, casual Instagram aesthetic.",
 "style": "photorealistic, candid, lifestyle, soft lighting, cozy aesthetic",
 "camera": {
 "type": "smartphone front camera",
 "angle": "mirror selfie, eye level",
 "lighting": "warm overhead light, soft shadows",
 "focus": "sharp on subject, slightly soft background"
 },
 "details": [
 "white cat interacting with subject",
 "messy mirror texture",
 "oversized streetwear",
 "muted color palette",
 "natural imperfections"
]
}
```

```
],
 "mood": "calm, intimate, casual",
 "resolution": "high"
}
```

I want you to recreate the above image but in a slight variation and with slightly different aesthetics. There should be more solar panels towards the bottom. I want you to give me an image prompt for a whisk or nano-baram in the above JSON style for a hero section of a solar company website. I'm going to further animate this, so please keep that in mind. It should have wind turbines and solar panels towards the bottom. I want you to give that to me. Thank you!